Contents

1 Proyecto Big Data KDD **4**
1.1 Entrega Unificada . . . 4

2 Paquete Final de Entrega **5**
2.1 Portada . . . 5
2.2 1. Documentos principales . . . 5
2.3 2. Alcance funcional entregado . . . 5
2.4 3. Checklist de revision previa a entrega . . . 6
2.4.1 3.1 Infraestructura . . . 6
2.4.2 3.2 Datos y pipeline . . . 6
2.4.3 3.3 Dashboard . . . 6
2.4.4 3.4 Documentacion . . . 6
2.5 4. Comandos de validacion rapida . . . 6
2.6 5. Nota final . . . 7

3 Resumen Ejecutivo - Sistema KDD Logistico **7**
3.1 Portada . . . 7
3.2 Indice . . . 7
3.3 1. Que se ha construido . . . 7
3.4 2. Arquitectura y flujo de valor . . . 8
3.4.1 Ingesta . . . 8
3.4.2 Procesamiento . . . 8
3.4.3 Almacenamiento y consulta . . . 8
3.4.4 Explotacion . . . 8
3.5 3. Resultados funcionales alcanzados . . . 9
3.6 4. Robustez y operaciones . . . 9
3.7 5. Estado tecnico final (entrega) . . . 9
3.8 6. Entregables documentales . . . 9
3.9 7. Conclusiones . . . 10

4 Memoria Tecnica del Sistema KDD Logistico **10**
4.1 Portada . . . 10
4.2 Indice . . . 10
4.3 1. Objetivo del sistema . . . 11
4.4 2. Arquitectura general . . . 11
4.4.1 Diagrama de arquitectura . . . 11
4.5 3. Flujo de datos end-to-end . . . 11
4.5.1 3.1 GPS . . . 11
4.5.2 3.2 Meteorologia . . . 12
4.5.3 3.3 Batch historico . . . 13
4.6 4. NiFi en detalle . . . 13
4.7 4.1 Estructura visual desplegada . . . 13
4.8 4.2 Controller Service . . . 13
4.9 4.3 Flujo GPS: procesadores y rol . . . 14
4.10 4.4 Flujo Clima: procesadores y rol . . . 15
4.11 4.5 Back-pressure y terminacion . . . 15
4.12 5. Generacion y sincronizacion raw . . . 16
4.13 5.1 GPS generator . . . 16

4.14 5.2 Raw HDFS loader . . . 16
4.15 6. Spark en detalle . . . 16
4.16 6.1 Preparacion comun . . . 17
4.17 6.2 Pipeline batch . . . 17
4.18 6.3 Pipeline streaming . . . 18
4.19 6.4 ML de riesgo de retraso . . . 19
4.20 7. Hive: tablas y vistas . . . 19
4.21 7.1 Tablas maestras y batch . . . 19
4.22 7.2 Tablas streaming . . . 19
4.23 7.3 Vistas en hora Madrid . . . 19
4.24 8. Cassandra . . . 20
4.25 9. Airflow en detalle . . . 20
4.26 10. Dashboard . . . 21
4.27 11. Orquestacion de arranque y reset . . . 23
4.28 12. Operacion y validacion recomendada . . . 24
4.29 13. Puntos de resiliencia implementados . . . 24
4.30 14. Limitaciones conocidas . . . 24

5 Flujo NiFi sugerido **24**
5.1 Portada . . . 24
5.2 Objetivo . . . 25
5.3 Flujo minimo (GPS) . . . 25
5.4 Recomendaciones . . . 26
5.5 Ingesta API meteorologica (requisito del enunciado) . . . 26
5.6 Generador GPS para NiFi . . . 27
5.7 Bootstrap automatico NiFi . . . 27

6 Dashboard Logistico **28**
6.1 Portada . . . 28
6.2 Indice . . . 28
6.3 Objetivo . . . 28
6.4 Captura actual . . . 28
6.5 Fuentes de datos del dashboard . . . 28
6.6 Que representa cada elemento visual . . . 30
6.6.1 Mapa en tiempo real . . . 30
6.6.2 Filtros de Tiempo Real (solo vista izquierda) . . . 30
6.6.3 Mapa de red logistica . . . 30
6.6.4 Tabla de ruta (debajo del mapa logistico) . . . 31
6.6.5 Insights de red (live) . . . 31
6.7 Consolidacion en Hive (reporting) . . . 31
6.8 Grafo por cercania geografica (k vecinos) . . . 32
6.8.1 Filtros de Analisis Logistico (solo vista derecha) . . . 32
6.9 Reglas de interpretacion de ruta por vehiculo . . . 32
6.10 ETA en panel de vehiculo . . . 32
6.11 Datos y nodos actuales . . . 33
6.12 Reinicio limpio de demo . . . 33
6.13 Nota de uso . . . 33

7 Operaciones del Proyecto KDD **33**
7.1 Portada . . . 33

7.2 Indice . . . 34
7.3 Arranque y parada . . . 34
7.4 Healthcheck en Airflow . . . 34
7.5 Arranque del stack desde Airflow . . . 35
7.6 Flujo NiFi . . . 36
7.7 Validaciones . . . 36
7.8 Politica de almacenamiento . . . 36
7.9 Consultas desde terminal (Hive y Cassandra) . . . 37
7.9.1 Hive (via spark-sql en `spark-client`) . . . 37
7.9.2 Cassandra (via `cqlsh` en contenedor `cassandra`) . . . 38
7.10 Sync de insights Cassandra -> Hive . . . 38
7.11 Matriz operativa end-to-end . . . 38
7.12 Limpieza de estado streaming . . . 39
7.13 Limpieza de historico failed en Airflow . . . 39
7.14 Zona horaria . . . 40
7.15 Vistas Hive en hora Madrid . . . 40
7.16 Bitacora operativa (actualizada 31/03/2026) . . . 40
7.17 Nota de tablas en tiempo real . . . 41
7.18 Operacion del dashboard (estado final) . . . 41
7.18.1 Desacople de vistas . . . 41
7.18.2 Reglas de filtro en Tiempo Real . . . 41
7.18.3 Fuente de ruta por vehiculo . . . 41
7.19 Troubleshooting dashboard . . . 41
7.19.1 Caso: veo ruta incoherente (`X -> X`) o destino inesperado . . . 41
7.19.2 Caso: ETA incoherente (ejemplo: >300 km y ETA en pocos minutos) . . . 42

8 Arquitectura del Proyecto **42**
8.1 Portada . . . 42
8.2 Indice . . . 42
8.3 1. Vista general . . . 43
8.4 2. Diagrama de arquitectura . . . 43
8.5 3. Componentes y responsabilidades . . . 44
8.6 4. Flujo de datos end-to-end . . . 45
8.7 5. Capas de almacenamiento . . . 45
8.8 6. Decision de desacople en dashboard . . . 45

9 Release Notes - 30/03/2026 **45**
9.1 Portada . . . 45
9.2 Indice . . . 45
9.3 Addendum - 31/03/2026 . . . 46
9.3.1 Dashboard . . . 46
9.3.2 Insights de red . . . 46
9.3.3 Spark / Hive . . . 46
9.3.4 Red y simulacion . . . 46
9.4 Resumen ejecutivo . . . 47
9.5 Dashboard web . . . 47
9.6 Datos de red y simulacion . . . 48
9.7 Airflow . . . 48
9.7.1 DAGs impactados . . . 48

9.7.2 Problema . . . 48
9.7.3 Solucion aplicada . . . 48
9.8 NiFi . . . 49
9.8.1 Bootstrap y visualizacion . . . 49
9.8.2 Limpieza de legacy . . . 49
9.9 Scripts nuevos/actualizados . . . 49
9.9.1 Nuevos . . . 49
9.9.2 Actualizados . . . 49
9.10 Verificaciones recomendadas . . . 50
9.10.1 Airflow . . . 50
9.10.2 NiFi . . . 50
9.11 Impacto operativo . . . 50

10 Proyecto Big Data KDD - Java 21 + Docker 50
10.1 Arquitectura . . . 51
10.2 Estructura . . . 51
10.3 Arranque rapido . . . 51
10.4 Scripts operativos . . . 52
10.5 Validacion automatica Hive . . . 53
10.6 Formato de datos (raw vs procesado) . . . 53
10.6.1 Matriz operativa (resumen) . . . 54
10.7 Vistas Hive en hora Madrid . . . 54
10.8 Servicios . . . 55
10.9 Cambios recientes (31/03/2026) . . . 56
10.10 Cambios recientes (30/03/2026) . . . 56
10.11 Notas . . . 57

11 Changelog 58
11.1 [Unreleased] . . . 58
11.1.1 Added . . . 58
11.1.2 Changed . . . 58
11.1.3 Fixed . . . 59
11.1.4 Removed . . . 59
11.2 [1.0.0] - 2026-03-30 . . . 59
11.2.1 Added . . . 59
11.2.2 Changed . . . 59
11.2.3 Fixed . . . 59
11.2.4 Removed . . . 59
11.2.5 Documentation . . . 59

1 Proyecto Big Data KDD

1.1 Entrega Unificada

Autor: Rául Meneses Gutiérrez

Curso: Curso de Especialización en Inteligencia Artificial y BigData

Fecha: 31/03/2026

2 Paquete Final de Entrega

2.1 Portada

- Proyecto: `Proyecto Big Data KDD - Logistica`
- Documento: `Indice maestro de entrega`
- Version: `v1.1`
- Fecha: `31/03/2026`

2.2 1. Documentos principales

1. Memoria tecnica completa:
   - docs/memoria-tecnica-sistema.md
2. Resumen ejecutivo (1-2 paginas):
   - docs/resumen-ejecutivo-memoria.md
3. Guia funcional del dashboard:
   - docs/dashboard.md
4. Guia de operaciones y troubleshooting:
   - docs/operations.md
5. Release notes de la iteracion:
   - docs/release-notes-2026-03-30.md
6. Guia general del repositorio:
   - README.md
7. Arquitectura tecnica + diagrama:
   - docs/architecture.md
   - docs/architecture-diagram.png
8. Guia de anotacion de codigo:
   - docs/CODE-ANNOTATION-GUIDE.md
9. Changelog consolidado:
   - CHANGELOG.md

2.3 2. Alcance funcional entregado

1. Plataforma KDD end-to-end con NiFi, Kafka, Spark, Hive, Cassandra, Airflow y dashboard web.
2. Ingesta separada por dominios (`gps_ingestion` y `weather_ingestion`) en NiFi.
   - PG por defecto actual: `kdd_ingestion_auto_v9`.
   - Archivado GPS sin sobrescritura por split (filename unico por evento).
3. Dashboard con dos vistas desacopladas:
   - Tiempo Real con filtros propios (`Origen RT`, `Destino RT`).
   - Red Logistica con filtros propios (`Origen`, `Destino`, `Perfil`).
4. Rutas y proyecciones de vehiculo coherentes con plan de trayecto (`planned_origin`, `planned_destination`).
5. Estado operativo con fuentes primarias en Cassandra y fallback controlado.
6. Insights de red live con historico persistido en Cassandra y consolidacion en Hive.
7. Red geografica consolidada (15 nodos) y flota ampliada (15 vehiculos).

2.4 3. Checklist de revision previa a entrega

2.4.1 3.1 Infraestructura

- ☐ `docker compose ps` sin servicios core caidos.
- ☐ NiFi accesible en `https://localhost:8443/nifi`.
- ☐ Airflow accesible en `http://localhost:8080`.
- ☐ Dashboard accesible en `http://localhost:8501`.

2.4.2 3.2 Datos y pipeline

- ☐ Topic Kafka GPS/Weather creados y activos.
- ☐ Raw en HDFS presente (`/data/raw/nifi/...`).
- ☐ Tablas Hive principales visibles en `transport_analytics`.
- ☐ Tabla de eventos enriquecidos streaming visible y con datos:
  - `transport_analytics.enriched_events_streaming`.
- ☐ Cassandra con estado de flota y observaciones meteo recientes.
- ☐ Tabla Cassandra de snapshots de insights con datos:
  - `transport.network_insights_snapshots`.
- ☐ Tablas Hive de insights consolidadas con datos:
  - `transport_analytics.network_insights_snapshots_hive`
  - `transport_analytics.network_insights_hourly_trends`.

2.4.3 3.3 Dashboard

- ☐ Modo oscuro por defecto y toggle claro/oscuro operativo.
- ☐ Filtros de Tiempo Real afectan solo vista izquierda.
- ☐ Filtros de Red Logistica afectan solo vista derecha.
- ☐ `TODOS -> TODOS` funciona en ambos contextos.
- ☐ No aparecen rutas espurias tipo `X -> X` en vehiculos.
- ☐ Panel de vehiculo muestra ruta, rumbo, siguiente nodo y ETA consistentes.
- ☐ Etiquetas de nodo RT en formato 3 letras y semitransparentes.
- ☐ Selectores origen/destino ordenados alfabeticamente.
- ☐ Tablas del dashboard ordenables por columna.

2.4.4 3.4 Documentacion

- ☐ Memoria tecnica y resumen ejecutivo actualizados.
- ☐ Dashboard/Operations/Release notes alineados con implementacion final.
- ☐ README consistente con comportamiento actual del sistema.

2.5 4. Comandos de validacion rapida

```
./scripts/start_kdd.sh
docker compose ps
curl -s http://localhost:8501/health
```

```
curl -s http://localhost:8501/api/debug/sources
./scripts/validate_hive_pipeline.sh
```

Nota:

- `start_kdd.sh` ya ejecuta validacion de compatibilidad Hive streaming y sanity check de vistas Madrid.

2.6 5. Nota final

Si se realizan cambios de frontend antes de defensa/demo, usar recarga fuerte del navegador:

`Ctrl+F5`

3 Resumen Ejecutivo - Sistema KDD Logistico

3.1 Portada

- Proyecto: `Proyecto Big Data KDD - Logistica`
- Documento: `Resumen ejecutivo para memoria`
- Version: `v1.1`
- Fecha: `31/03/2026`

3.2 Indice

1. Que se ha construido
2. Arquitectura y flujo de valor
3. Resultados funcionales alcanzados
4. Robustez y operaciones
5. Estado tecnico final (entrega)
6. Entregables documentales
7. Conclusiones

Este resumen sintetiza la solucion implementada para la entrega del proyecto, con foco en arquitectura, valor funcional, robustez operativa y estado final.

3.3 1. Que se ha construido

Se ha desarrollado una plataforma Big Data end-to-end para analisis logistico que integra:

1. Ingesta de eventos GPS y meteorologia en tiempo casi real.
2. Procesamiento batch y streaming sobre Spark.
3. Analitica de red logistica con grafos y rutas optimizadas.
4. Scoring de riesgo de retraso mediante ML.
5. Persistencia en HDFS/Hive y estado operativo en Cassandra.
6. Orquestacion y monitorizacion con Airflow.
7. Dashboard web operativo para explotacion funcional.

El sistema esta empaquetado en Docker Compose, con scripts de arranque, validacion y limpieza para reproducibilidad local.

3.4 2. Arquitectura y flujo de valor

3.4.1 Ingesta

- NiFi organiza la ingesta en dos subflujos claramente separados:
  - `gps_ingestion`
  - `weather_ingestion`
- Publica en Kafka cuatro topics principales:
  - `transport.raw`
  - `transport.filtered`
  - `transport.weather.raw`
  - `transport.weather.filtered`
- Conserva raw local y un proceso dedicado (`raw-hdfs-loader`) lo sincroniza a HDFS para trazabilidad y reprocesado.

3.4.2 Procesamiento

- Spark batch construye tablas historicas y analiticas:
  - enriquecimiento de eventos,
  - agregados de delay/velocidad,
  - metricas de grafo (`connected components`, `pagerank`),
  - shortest paths,
  - scoring ML de riesgo.
- Spark streaming procesa GPS y clima filtrado desde Kafka, con watermarks y deduplicacion, y persiste resultados en Hive.
- Spark streaming persiste ademas eventos enriquecidos en tiempo real en `transport_analytics.enriched_ever`

3.4.3 Almacenamiento y consulta

- Hive centraliza el catalogo analitico (batch y streaming).
- Cassandra mantiene el ultimo estado de cada vehiculo para consulta rapida.
- Cassandra almacena snapshots de insights de red para historico.
- HDFS mantiene capa raw y curated.

3.4.4 Explotacion

- Dashboard con:
  - mapa operativo de flota en tiempo real,
  - analisis de red logistica y mejor ruta por perfil,
  - impacto meteorologico en ETA y penalizacion de ruta.
- El dashboard usa Cassandra como fuente principal de flota y fallback a archivos NiFi si Cassandra no esta disponible.

- El dashboard usa plan de trayecto por vehiculo (`planned_origin`, `planned_destination`) para reforzar coherencia de rutas visualizadas.
- Los filtros de Tiempo Real y de Red Logistica estan desacoplados para evitar interferencias entre vistas.

3.5 3. Resultados funcionales alcanzados

1. Visualizacion de vehiculos con trazas y foco por seleccion.
2. Calculo de ETA por vehiculo con suavizado para evitar fluctuaciones bruscas.
3. Calculo de mejor ruta por perfiles (`balanced`, `fastest`, `resilient`).
4. Incorporacion real del factor meteorologico en estimaciones de tiempo.
5. Representacion tabular de rutas y metricas (distancia, tiempos, penalizacion clima, factor aplicado).
6. Modo oscuro por defecto y cambio claro/oscuro en frontend.
7. Filtros RT parciales y completos (`TODOS->X`, `X->TODOS`, `X->Y`) con limpieza de seleccion fuera de filtro.
8. Insights live de red (cuellos de botella + nodos criticos + historico).
9. Tablas ordenables y selectores alfabeticos en ambas vistas.

3.6 4. Robustez y operaciones

Se incorporaron mejoras de estabilidad relevantes para entrega:

1. Healthchecks Airflow (diario y horario) con degradacion segura en consultas smoke, evitando falsos fallos cuando las tablas fuente streaming no estan disponibles temporalmente.
2. Limpieza controlada de historico fallido en Airflow para dejar estado operativo en verde.
3. Reorganizacion visual de NiFi por dominios (GPS/Clima), simplificando soporte y revisiones.
4. Script de limpieza de Process Groups legacy de NiFi para mantener entorno limpio.
5. Fallback Parquet en Spark streaming si falla escritura a Hive, evitando parada del pipeline.

3.7 5. Estado tecnico final (entrega)

- NiFi:
  - Process Group activo: `kdd_ingestion_auto_v9`
  - Subgrupos: `gps_ingestion`, `weather_ingestion`
- Airflow:
  - DAGs de healthcheck estabilizados y sin failed pendientes en la validacion operativa.
- Dashboard:
  - Endpoint de diagnostico de fuentes: `/api/debug/sources`
  - Fuente vehiculos: Cassandra (primaria), `nifi/input` (fallback).
  - Endpoint de historico insights: `/api/network/insights/history`.
  - Red consolidada en 15 nodos y flota de 15 vehiculos (14 activos + 1 mantenimiento).

3.8 6. Entregables documentales

Se han dejado documentos para operacion y defensa:

1. Memoria tecnica completa:
   - `docs/memoria-tecnica-sistema.md`
2. Operaciones y checklist:
   - `docs/operations.md`
3. Dashboard y leyenda funcional:
   - `docs/dashboard.md`
4. Release notes de iteracion:
   - `docs/release-notes-2026-03-30.md`

3.9 7. Conclusiones

La solucion cumple los objetivos del proyecto KDD en un entorno reproducible y demostrable:

- separa correctamente ingesta, transformacion, analitica y visualizacion,
- combina batch + streaming con gobernanza de datos,
- incorpora analitica avanzada (grafos y ML),
- ofrece mecanismos de observabilidad, resiliencia y mantenimiento.

Como resultado, se dispone de un sistema funcional para analisis logistico en tiempo real y soporte de decisiones operativas.

4 Memoria Tecnica del Sistema KDD Logistico

4.1 Portada

- Proyecto: `Proyecto Big Data KDD - Logistica`
- Documento: `Memoria tecnica del sistema`
- Version: `v1.1`
- Fecha: `31/03/2026`

4.2 Indice

1. Objetivo del sistema
2. Arquitectura general
3. Flujo de datos end-to-end
4. NiFi en detalle
5. Generacion y sincronizacion raw
6. Spark en detalle
7. Hive: tablas y vistas
8. Cassandra
9. Airflow en detalle
10. Dashboard
11. Orquestacion de arranque y reset
12. Operacion y validacion recomendada
13. Puntos de resiliencia implementados

14. Limitaciones conocidas

Documento de entrega con explicacion detallada del funcionamiento extremo a extremo del sistema: ingesta, procesamiento batch/streaming, almacenamiento, orquestacion y visualizacion.

Resumen ejecutivo asociado (1-2 paginas):

- `docs/resumen-ejecutivo-memoria.md`

4.3 1. Objetivo del sistema

El sistema implementa un ciclo KDD para analitica logistica, combinando:

- ingesta continua de GPS y meteorologia,
- procesamiento batch y streaming con Spark,
- analitica de red con grafos (GraphFrames),
- scoring de riesgo de retraso con ML,
- persistencia en Hive/HDFS y Cassandra,
- monitorizacion/orquestacion con Airflow,
- visualizacion web en dashboard.

4.4 2. Arquitectura general

Componentes principales del `docker-compose`:

1. `kafka` (Apache Kafka 3.9.1): bus de eventos.
2. `nifi` (Apache NiFi 2.7.0): ingesta y enrutamiento.
3. `gps-generator`: generador de eventos GPS simulados.
4. `hadoop`: HDFS + YARN (nodo unico pseudo-distribuido).
5. `raw-hdfs-loader`: sincroniza raw de NiFi hacia HDFS.
6. `hive-metastore` + `hive-server`: catalogo y consulta SQL.
7. `spark-client`: ejecucion de jobs Spark (batch y streaming).
8. `cassandra`: estado mas reciente por vehiculo para baja latencia.
9. `airflow-webserver` + `airflow-scheduler` + `airflow-postgres`: orquestacion.
10. `dashboard`: API HTTP + frontend para operacion y analitica visual.

4.4.1 Diagrama de arquitectura

Archivo fuente del diagrama:

- `docs/architecture-diagram.png`

4.5 3. Flujo de datos end-to-end

4.5.1 3.1 GPS

1. `gps-generator` crea ficheros `gps_*.jsonl` en `nifi/input`.
2. NiFi lee esos ficheros, extrae atributos y publica:
    - `transport.raw` (todos los eventos),

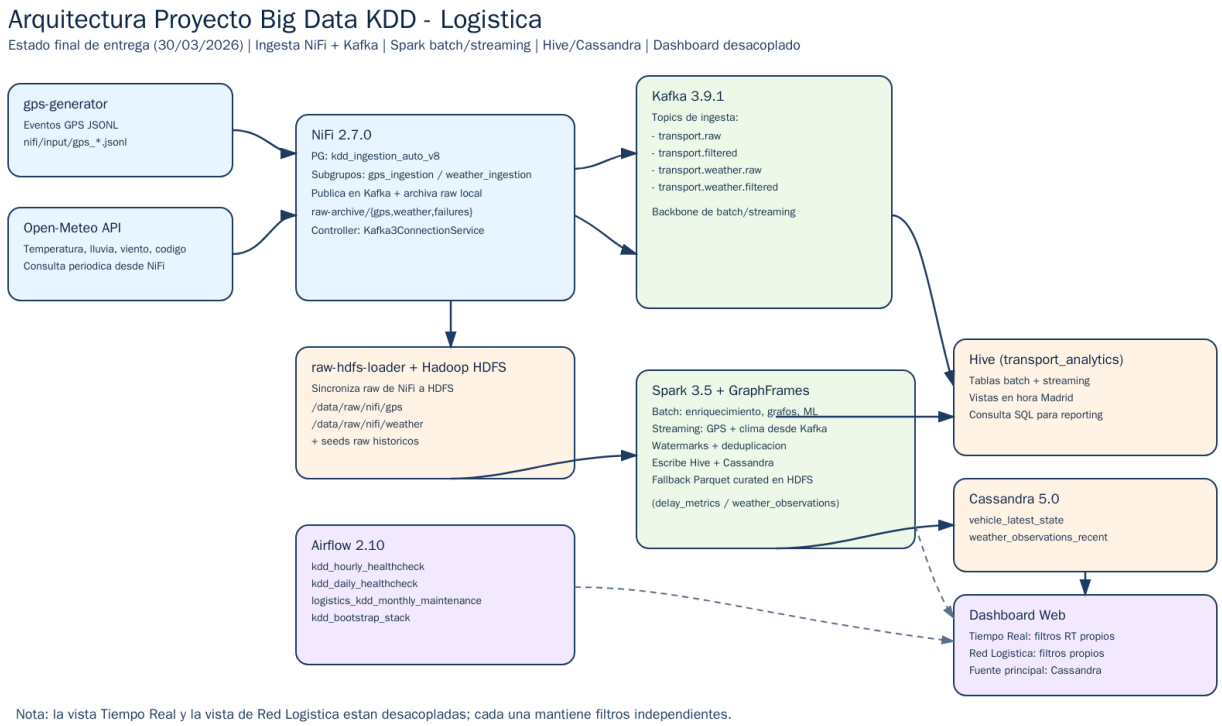

Figure 1: Diagrama de arquitectura del proyecto

- `transport.filtered` (eventos con `delay_minutes >= 5`).

3. NiFi archiva raw local en `nifi/raw-archive/gps`.
4. `raw-hdfs-loader` mueve esos raw a HDFS: `/data/raw/nifi/gps/...`.
5. Spark streaming consume `transport.filtered`, agrega ventanas y escribe:
   - `transport_analytics.delay_metrics_streaming` (Hive),
   - `transport_analytics.enriched_events_streaming` (Hive),
   - fallback Parquet: `/data/curated/delay_metrics_streaming`.

4.5.2 3.2 Meteorologia

1. NiFi consulta Open-Meteo periodicamente.
2. NiFi extrae y normaliza campos de clima.
3. NiFi publica:
   - `transport.weather.raw`,
   - `transport.weather.filtered`.
4. NiFi archiva raw local en `nifi/raw-archive/weather`.
5. `raw-hdfs-loader` sincroniza a HDFS: `/data/raw/nifi/weather/...`.
6. Spark streaming consume `transport.weather.filtered` y escribe:
   - vista operativa de consulta: `transport_analytics.v_weather_observations_madrid`,
   - fallback Parquet: `/data/curated/weather_observations_streaming`.

4.5.3 3.3 Batch historico

1. Spark batch lee `hdfs://hadoop:9000/data/raw/gps_events.jsonl`.
2. Limpia/normaliza y enriquece con maestros de almacenes/vehiculos.
3. Calcula agregados, grafos y modelo ML.
4. Publica tablas analiticas en Hive.

4.6 4. NiFi en detalle

4.7 4.1 Estructura visual desplegada

Capturas de referencia de los subflujos:

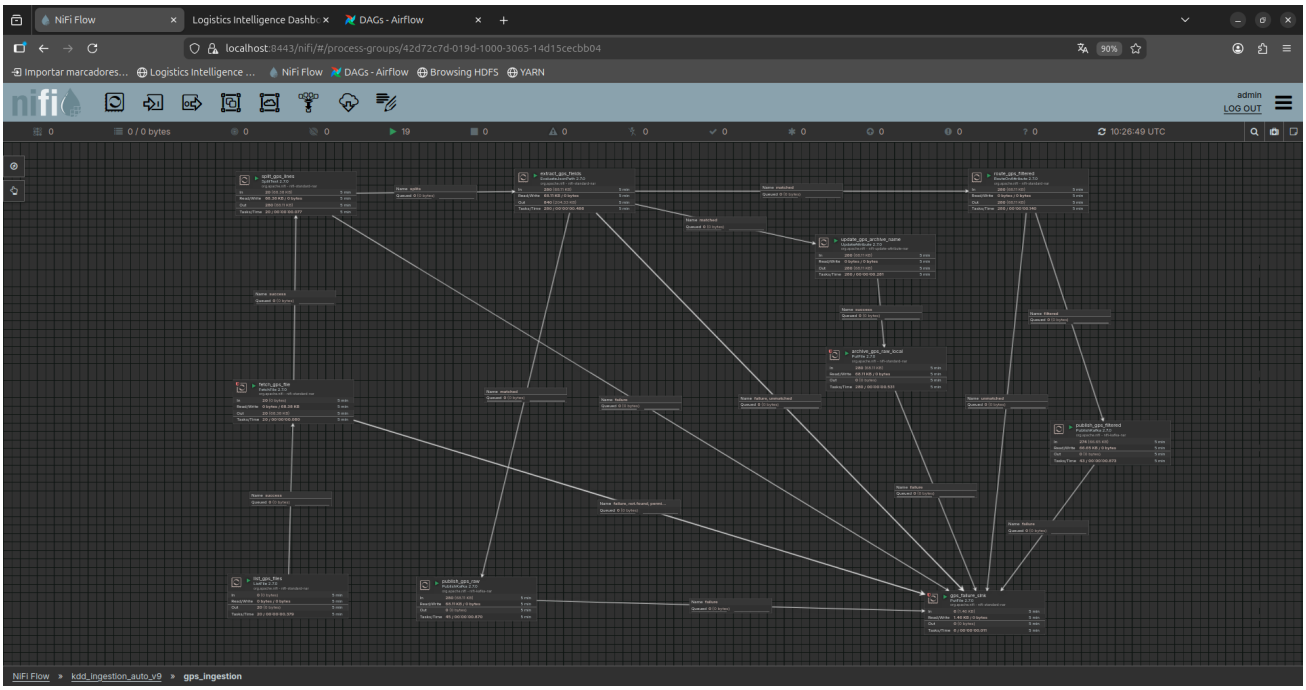

Figure 2: NiFi GPS ingestion

Bootstrap actual (`scripts/bootstrap_nifi_flow.sh`) crea:

- Process Group principal: `kdd_ingestion_auto_v9`
- Subgrupos:
  - `gps_ingestion`
  - `weather_ingestion`

Esta separacion facilita mantenimiento y lectura en la UI.

4.8 4.2 Controller Service

- `kafka3_connection_service` (`Kafka3ConnectionService`)
  - `bootstrap.servers = kafka:9092`
  - compartido por procesadores `PublishKafka`.

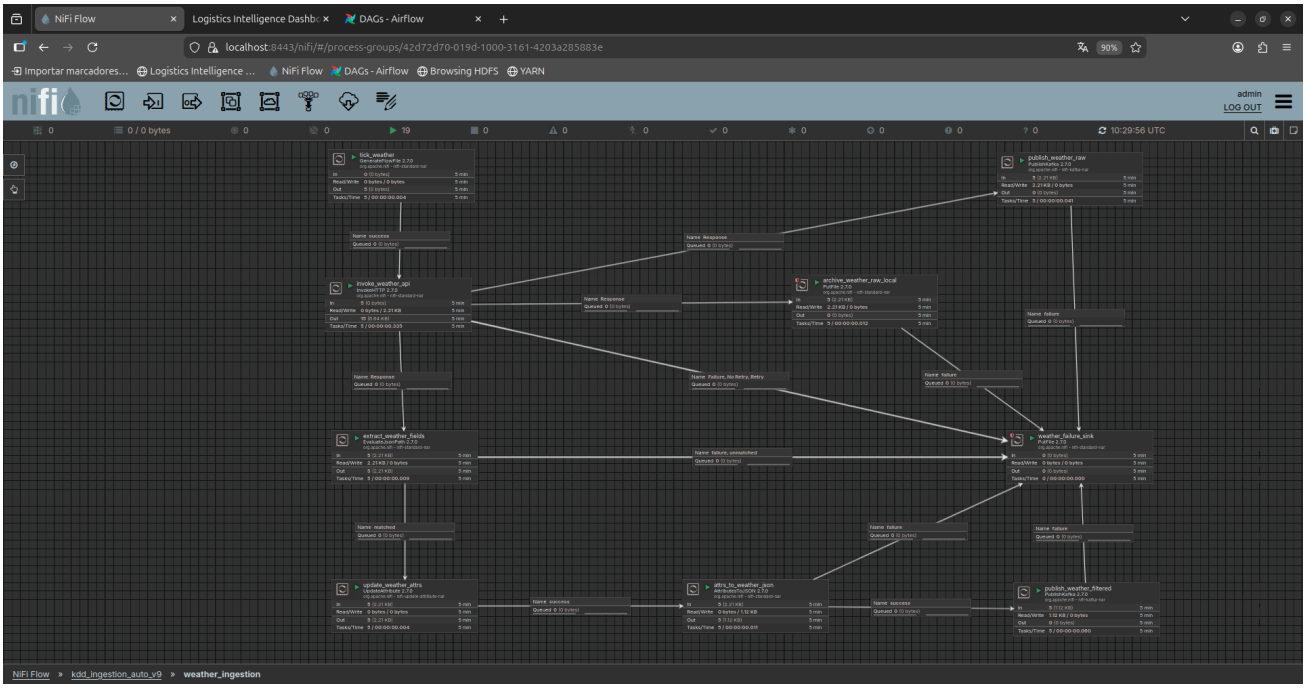

Figure 3: NiFi WEATHER ingestion

4.9 4.3 Flujo GPS: procesadores y rol

1. `list_gps_files` (`ListFile`)
   - dir: `/opt/nifi/nifi-current/input`
   - filtro: `.*\.jsonl`
   - periodo: `15 sec`.
2. `fetch_gps_file` (`FetchFile`)
3. `split_gps_lines` (`SplitText`)
   - `Line Split Count=1`.
4. `extract_gps_fields` (`EvaluateJsonPath`)
   - extrae: `vehicle_id`, `warehouse_id`, `event_type`, `delay_minutes`.
5. `update_gps_archive_name` (`UpdateAttribute`)
   - `filename = ${filename}_${fragment.index}_${UUID()}.jsonl`.
6. `route_gps_filtered` (`RouteOnAttribute`)
   - regla `filtered`: `${delay_minutes:toNumber():ge(5)}`.
7. `publish_gps_raw` (`PublishKafka`)
   - topic: `transport.raw`.
8. `publish_gps_filtered` (`PublishKafka`)
   - topic: `transport.filtered`.
9. `archive_gps_raw_local` (`PutFile`)
   - dir: `/opt/nifi/nifi-current/raw-archive/gps`.
10. `gps_failure_sink` (`PutFile`)

- dir: `/opt/nifi/nifi-current/raw-archive/failures/gps`.

Ruteo principal:

- `matched` de `extract_gps_fields` pasa por `update_gps_archive_name` antes de archivado local (evita sobrescritura por split).
- `filtered` de `route_gps_filtered` va a Kafka filtrado.
- Errores y `unmatched` van a `gps_failure_sink`.

4.10 4.4 Flujo Clima: procesadores y rol

1. `tick_weather` (`GenerateFlowFile`)
   - periodo: `60 sec`.
2. `invoke_weather_api` (`InvokeHTTP`)
   - URL Open-Meteo (Madrid).
3. `extract_weather_fields` (`EvaluateJsonPath`)
   - `temperature_c`, `precipitation_mm`, `wind_kmh`, `weather_code`.
4. `update_weather_attrs` (`UpdateAttribute`)
   - `weather_event_id=${UUID()}`
   - `warehouse_id=WH1`
   - `source=open-meteo`
   - `observation_time` en UTC.
5. `attrs_to_weather_json` (`AttributesToJSON`)
   - transforma atributos a JSON final.
6. `publish_weather_raw` (`PublishKafka`)
   - topic: `transport.weather.raw`.
7. `publish_weather_filtered` (`PublishKafka`)
   - topic: `transport.weather.filtered`.
8. `archive_weather_raw_local` (`PutFile`)
   - dir: `/opt/nifi/nifi-current/raw-archive/weather`.
9. `weather_failure_sink` (`PutFile`)
   - dir: `/opt/nifi/nifi-current/raw-archive/failures/weather`.

Ruteo principal:

- `Response` de `InvokeHTTP` alimenta raw Kafka, archivo raw y extraccion.
- Camino normal termina en `publish_weather_filtered`.
- `Failure/Retry/No Retry` y fallos de transformacion/publicacion van a `weather_failure_sink`.

4.11 4.5 Back-pressure y terminacion

Cada conexion creada por bootstrap aplica:

- `backPressureObjectThreshold = 10000`
- `backPressureDataSizeThreshold = 1 GB`
- `flowFileExpiration = 0 sec`

Ademas, se auto-terminan relaciones no usadas para evitar bloqueos por conexiones obligatorias sin destino.

4.12 5. Generacion y sincronizacion raw

4.13 5.1 GPS generator

Script: `scripts/gps_generator.py`.

Caracteristicas:

- Frecuencia: cada `15s`.
- Factor temporal simulado: `SIM_TIME_FACTOR=6`.
- Flota cargada desde `data/master/vehicles.csv` (IDs actuales `TRUCK-*`, filtrando por `status` activo), 1 evento por vehiculo y ciclo.
- Catalogo de rutas realistas calculado desde el grafo logistico y reasignacion dinamica al cerrar tramos.
- Movimiento progresivo entre ciudades (sin salto directo idealizado).
- Estado persistido en `.vehicle_path_state.json` para continuidad de trayectorias.

Campos emitidos por evento:

- `event_id`, `vehicle_id`, `warehouse_id`, `route_id`, `event_type`,
- `latitude`, `longitude`, `delay_minutes`, `speed_kmh`, `event_time`.

4.14 5.2 Raw HDFS loader

Script: `docker/hadoop/scripts/load-nifi-raw-to-hdfs.sh`.

Funcion:

- Escanea raw local de NiFi (`/opt/nifi-raw/gps`, `/opt/nifi-raw/weather`) cada 30s.
- Sube ficheros a `/data/raw/nifi/{gps|weather}` en HDFS.
- Mueve ficheros procesados a `.processed` local para evitar reprocesado.

4.15 6. Spark en detalle

Clase principal: `com.proyectobigdata.LogisticsAnalyticsJob`.

Modos:

- `batch` (job historico + grafos + ML)
- `streaming` (consumo continuo de Kafka)
- `insights-sync` (consolidacion de snapshots de insights Cassandra -> Hive)

Lanzadores:

- `spark-app/run-batch.sh`
- `spark-app/run-streaming.sh`
- `spark-app/run-insights-sync.sh`

Paquetes Spark usados:

- `spark-sql-kafka-0-10_2.12:3.5.5`
- `graphframes-spark3_2.12:0.9.0-spark3.5`

- `spark-cassandra-connector_2.12:3.5.1`

4.16 6.1 Preparacion comun

Al iniciar:

1. Configura Spark SQL timezone (`SPARK_SQL_TIMEZONE`, por defecto `Europe/Madrid`).
2. Habilita soporte Hive (`enableHiveSupport`).
3. Crea DB Hive `transport_analytics` si no existe.
4. Carga maestros Hive:
   - `transport_analytics.master_warehouses`
   - `transport_analytics.master_vehicles`
5. Inicializa esquema Cassandra:
   - keyspace `transport`
   - tabla `transport.vehicle_latest_state`.

4.17 6.2 Pipeline batch

Entrada:

- `hdfs://hadoop:9000/data/raw/gps_events.jsonl`

Pasos:

1. Limpieza y normalizacion (`cleanAndNormalizeEvents`):
   - `event_time -> event_timestamp`,
   - cast de `delay_minutes`, `speed_kmh`,
   - filtros de nulos clave.
2. Deduplicacion por `event_id`.
3. Enriquecimiento con maestros (`warehouse` + `vehicle`).
4. Persistencia:
   - Parquet curated: `/data/curated/enriched_events`
   - Hive: `transport_analytics.enriched_events`.
5. Agregacion por ventana de 15 min y almacen:
   - Hive: `transport_analytics.delay_metrics_batch`.
6. Grafo (`GraphFrames`):
   - connected components + pagerank
   - Hive: `transport_analytics.route_graph_metrics`.
7. Shortest paths con landmarks de almacenes criticos:
   - Hive: `transport_analytics.route_shortest_paths`.
8. Estado mas reciente por vehiculo a Cassandra.
9. Entrenamiento/scoring ML de retraso:
   - Hive: `transport_analytics.ml_delay_risk_scores`
   - Modelo: `hdfs://hadoop:9000/models/delay_risk_rf`.

4.18 6.3 Pipeline streaming

Entradas Kafka:

- `transport.filtered` (GPS filtrado)
- `transport.weather.filtered` (meteo filtrada)

Pasos GPS streaming:

1. Parse JSON desde `value`.
2. Limpieza y normalizacion.
3. Watermark de `20 minutes`.
4. Deduplicacion por `event_id`.
5. Enriquecimiento con maestros.
6. Ventana de 15 min por `warehouse_id`.
7. Escritura `foreachBatch` a Hive:
   - `transport_analytics.delay_metrics_streaming`.
   - `transport_analytics.enriched_events_streaming`.
8. Escritura de latest state por vehiculo a Cassandra.

Pasos clima streaming:

1. Parse de campos meteo desde JSON.
2. Limpieza y normalizacion (`weather_timestamp`, casts, filtros).
3. Watermark `20 minutes`.
4. Deduplicacion por `weather_event_id`.
5. Escritura `foreachBatch` a Hive:
   - consumo operativo via vista `transport_analytics.v_weather_observations_madrid`.

Fallback robustez:

- Si falla escritura Hive (incompatibilidad/metastore), guarda en Parquet fallback:
  - `/data/curated/delay_metrics_streaming`
  - `/data/curated/enriched_events_streaming`
  - `/data/curated/weather_observations_streaming`

Checkpoints:

- `/tmp/checkpoints/delay_metrics`
- `/tmp/checkpoints/latest_vehicle_state`
- `/tmp/checkpoints/weather_observations`

Compatibilidad operativa:

- El script `scripts/ensure_hive_streaming_compat.sh` asegura que existan:
  - `transport_analytics.delay_metrics_streaming`
  - `transport_analytics.weather_observations_streaming`
  - vistas Madrid asociadas.
- Se ejecuta en arranque para evitar vistas “colgadas” tras limpiezas.

4.19 6.4 ML de riesgo de retraso

Modelo principal:

- `RandomForestRegressor`
- features:
  - `speed_kmh`
  - `warehouse_id` (index + one-hot)
  - `vehicle_type` (index + one-hot)
- label: `delay_minutes`

Logica:

- si dataset >= 3 filas: entrena modelo (con split 80/20 si >=20).
- evalua RMSE en test.
- puntua ultimo estado por vehiculo y clasifica riesgo:
  - `high` (>=10)
  - `medium` (>=5)
  - `low` (<5)

Fallback heuristico (si dataset pequeno):

- regla por `speed_kmh` para `predicted_delay_minutes`.

4.20 7. Hive: tablas y vistas

4.21 7.1 Tablas maestras y batch

- `transport_analytics.master_warehouses`
- `transport_analytics.master_vehicles`
- `transport_analytics.enriched_events`
- `transport_analytics.delay_metrics_batch`
- `transport_analytics.route_graph_metrics`
- `transport_analytics.route_shortest_paths`
- `transport_analytics.ml_delay_risk_scores`

4.22 7.2 Tablas streaming

- `transport_analytics.delay_metrics_streaming`
- `transport_analytics.enriched_events_streaming`
- (opcional segun entorno) `transport_analytics.weather_observations_streaming`
- `transport_analytics.network_insights_snapshots_hive`
- `transport_analytics.network_insights_hourly_trends`

4.23 7.3 Vistas en hora Madrid

Vistas usadas por healthchecks/dashboard:

- `transport_analytics.v_weather_observations_madrid`

- `transport_analytics.v_delay_metrics_streaming_madrid`

Nota operativa:

- Si las tablas streaming fuente no estan disponibles temporalmente, los DAGs de healthcheck degradan a validaciones seguras para no producir falsos rojos.
- En esta entrega, para consultas meteo de operacion se usa `transport_analytics.v_weather_observations_m`

4.24 8. Cassandra

Keyspace/tablas:

- `transport.vehicle_latest_state`
- `transport.weather_observations_recent`
- `transport.network_insights_snapshots`

Uso:

- mantener ultimo estado por `vehicle_id` para consulta rapida.

Campos:

- `vehicle_id` (PK), `warehouse_id`, `route_id`,
- `last_event_timestamp`, `delay_minutes`, `speed_kmh`,
- `latitude`, `longitude`.

4.25 9. Airflow en detalle

Captura de referencia del entorno Airflow:

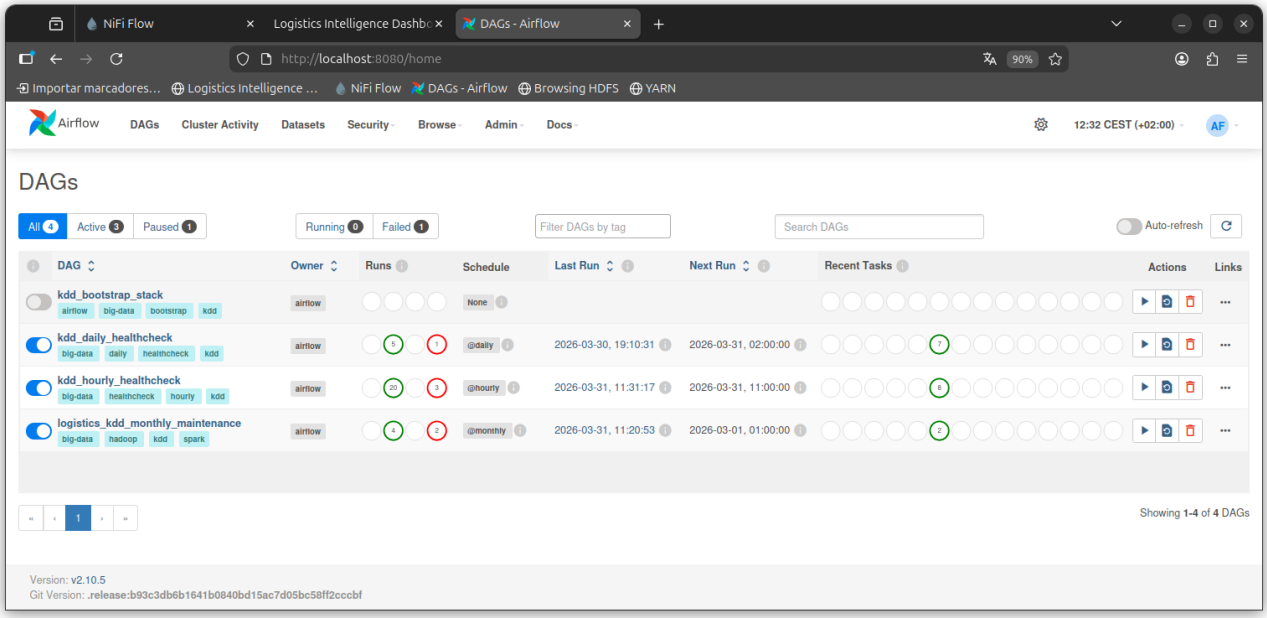

Figure 4: Airflow

DAGs:

1. `kdd_bootstrap_stack`
   - Manual.
   - Arranca servicios no-Airflow con `docker compose up`.
2. `kdd_hourly_healthcheck` (`@hourly`)
3. `kdd_daily_healthcheck` (`@daily`)
4. `logistics_kdd_monthly_maintenance` (`@monthly`)
   - ejecuta batch Spark y limpia checkpoints temporales.

Healthchecks (hourly/daily):

- contenedores core,
- topics Kafka obligatorios,
- rutas HDFS,
- tablas Hive base,
- vistas Madrid,
- smoke query sobre vistas (con fallback seguro).

Alertas:

- `on_failure_callback` en todos los DAGs,
- opcional email via `AIRFLOW_ALERT_EMAIL`.

4.26 10. Dashboard

Captura de estado operativo del dashboard:

Backend:

- `dashboard/server.py` (HTTP server multihilo).
- Lee:
  - estado de vehiculo desde Cassandra (`transport.vehicle_latest_state`) como fuente primaria,
  - fallback de GPS desde `nifi/input` si Cassandra no esta disponible,
  - plan de trayecto por vehiculo desde `nifi/input/.vehicle_path_state.json` (`planned_origin`, `planned_destination`),
  - clima reciente desde Cassandra (`transport.weather_observations_recent`) como fuente primaria,
  - fallback de clima con snapshots en `nifi/raw-archive/weather`,
  - grafo y maestros desde `data/graph/*.csv`, `data/master/warehouses.csv`.

API principal:

- `/api/overview`
- `/api/vehicles/latest`
- `/api/vehicles/history`
- `/api/weather/latest`

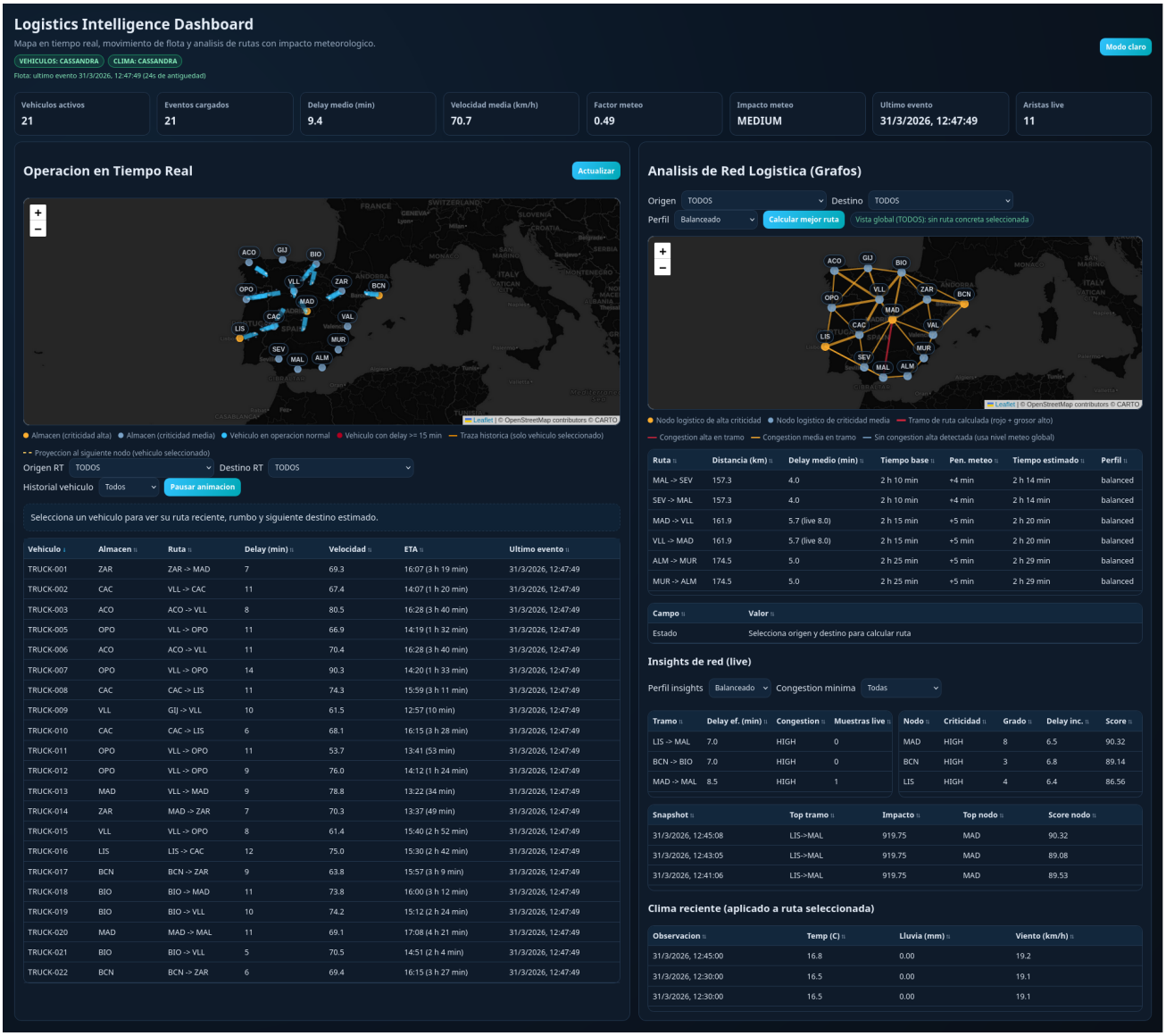

Figure 5: Dashboard logistica (21 vehiculos activos)

- `/api/network/graph`
- `/api/network/best-route`
- `/api/network/insights/history`
- `/api/debug/sources` (diagnostico de fuentes activas y fallback)

Motor de ruta:

- Dijkstra en backend con perfiles (`balanced`, `fastest`, `resilient`).
- Peso de arista considera distancia, delay promedio e impacto meteo.

Frontend:

- modo oscuro por defecto (toggle claro/oscuro),
- mapa tiempo real y mapa de red logistica,
- tabla de rutas con ETA y desglose de tiempos.
- tablas ordenables por columna.
- filtros desacoplados por vista:
  - Tiempo Real: `Origen RT` / `Destino RT` (solo columna izquierda),
  - Red Logistica: `Origen` / `Destino` / `Perfil` (solo columna derecha).
- comportamiento de filtros RT:
  - `TODOS->TODOS` vista global,
  - filtros parciales (`TODOS->X` o `X->TODOS`) soportados.
- reglas de consistencia:
  - no ruta concreta con origen=destino en bloque logistico,
  - limpieza automatica de seleccion de vehiculo si deja de cumplir filtro RT,
  - priorizacion de `planned_origin/planned_destination` para evitar rutas `X->X` espurias y proyecciones incoherentes.
  - selectores de origen/destino ordenados alfabeticamente en ambas vistas,
  - etiquetas de nodo RT en formato 3 letras y semitransparentes para no ocultar vehiculos.

4.27 11. Orquestacion de arranque y reset

Script principal de arranque:

- `scripts/start_kdd.sh`
  - levanta stack,
  - espera NiFi,
  - ejecuta bootstrap NiFi (por defecto `kdd_ingestion_auto_v9`),
  - verifica dashboard.

Reset de demo:

- `scripts/reset_demo_data.sh`
  - limpia eventos locales,
  - resetea estado streaming (Hive + checkpoints),
  - recrea generadores.

4.28 12. Operacion y validacion recomendada

Arranque:

```
./scripts/start_kdd.sh
```

Validacion E2E batch+streaming:

```
./scripts/validate_hive_pipeline.sh
```

Limpieza runs rojos historicos (healthchecks):

```
./scripts/cleanup_airflow_failed_runs.sh --apply
```

Limpieza de Process Groups legacy NiFi:

```
./scripts/cleanup_nifi_legacy_pgs.py
```

4.29 13. Puntos de resiliencia implementados

1. Fallback a Parquet cuando Hive streaming falla.
2. Manejo de excepciones en escritura Cassandra (no bloquea pipeline).
3. Healthchecks Airflow con degradacion segura en smoke queries.
4. Colas de error separadas en NiFi (`failure sinks` GPS/Clima).
5. Separacion visual de flujos NiFi para diagnostico rapido.

4.30 14. Limitaciones conocidas

1. El dashboard depende de Cassandra para `/api/vehicles/latest`; si Cassandra cae, usa fallback desde `nifi/input` y puede perderse parte del estado reciente.
2. Si Cassandra no esta disponible, la precision de clima en dashboard pasa a depender del fallback de snapshots en `raw-archive/weather`.
3. En entorno local, tiempos de arranque de NiFi/Spark pueden variar segun recursos.

Esta memoria describe el estado tecnico actual del proyecto para entrega academica y operacion reproducible del entorno.

5 Flujo NiFi sugerido

5.1 Portada

- Proyecto: `Proyecto Big Data KDD - Logistica`
- Documento: `Flujo NiFi operativo`
- Version: `v1.0-entrega`
- Fecha: `30/03/2026`

5.2 Objetivo

Crear una ingesta dual hacia Kafka y HDFS:

- tema `transport.raw`
- tema `transport.filtered`
- tema `transport.weather.raw`
- tema `transport.weather.filtered`
- copia raw en HDFS

5.3 Flujo minimo (GPS)

Captura del subflujo desplegado en NiFi:

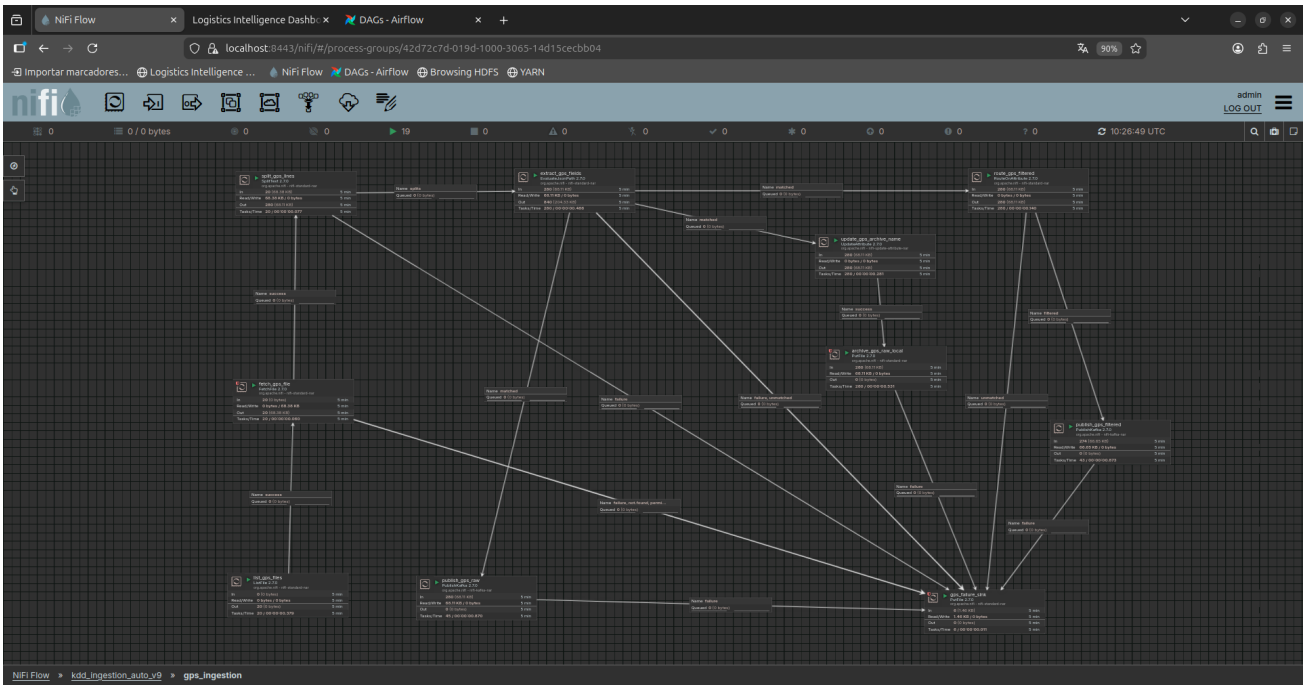

Figure 6: NiFi GPS ingestion

1. `ListFile`
   - Directorio: `/opt/nifi/nifi-current/input`
   - Filtro: `*.jsonl`
2. `FetchFile`
3. `SplitText`
   - 1 linea por evento
4. `EvaluateJsonPath`
   - extraer `vehicle_id`, `warehouse_id`, `event_type`, `delay_minutes`
5. `UpdateAttribute`
   - `filename = ${filename}_${fragment.index}_${UUID()}.jsonl`
   - objetivo: evitar sobrescritura en `raw-archive/gps` cuando `SplitText` genera multiples flowfiles desde un mismo archivo.

6. `RouteOnAttribute`
   - `filtered: ${delay_minutes:toNumber():ge(5)}`
7. `PublishKafka` (NiFi 2.7 con `Kafka3ConnectionService`)
   - Topic raw (todos los GPS): `transport.raw`
8. `PublishKafka` (NiFi 2.7 con `Kafka3ConnectionService`)
   - Topic filtered: `transport.filtered`
9. `PutFile`
   - Directorio: `/opt/nifi/nifi-current/raw-archive/gps`
10. `raw-hdfs-loader`

- Sincroniza `nifi/raw-archive` -> HDFS `/data/raw/nifi`

5.4 Recomendaciones

- Habilitar `back pressure` en las conexiones NiFi.
- Configurar relaciones `failure` hacia una cola de reintentos.
- En el bootstrap actual se enrutan errores a:
  - `/opt/nifi/nifi-current/raw-archive/failures/gps`
  - `/opt/nifi/nifi-current/raw-archive/failures/weather`

5.5 Ingesta API meteorologica (requisito del enunciado)

Captura del subflujo de meteorologia desplegado en NiFi:

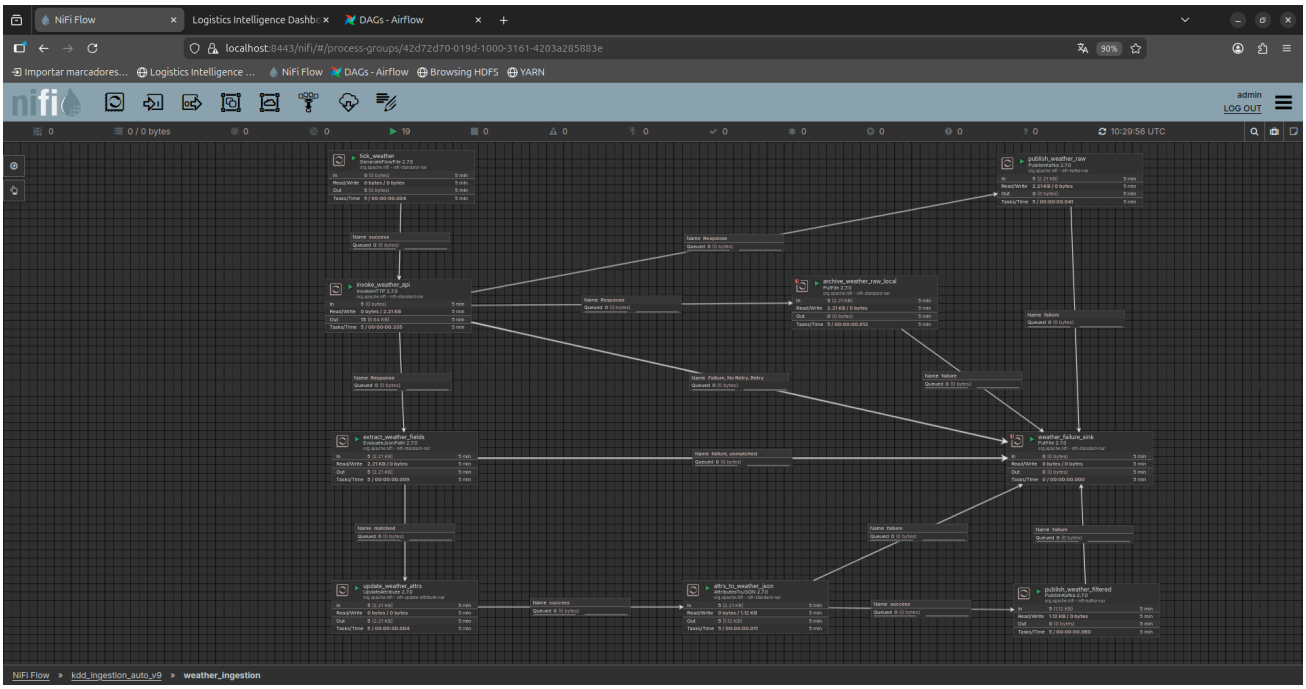

Figure 7: NiFi WEATHER ingestion

Flujo adicional recomendado para clima:

1. `GenerateFlowFile`

    - Scheduling: cada 60s
2. `InvokeHTTP`
    - Metodo: `GET`
    - URL API publica (Open-Meteo):
    - `https://api.open-meteo.com/v1/forecast?latitude=40.4168&longitude=-3.7038&current=tempe`
3. `EvaluateJsonPath`
    - Extraer `temperature_2m`, `precipitation`, `wind_speed_10m`
4. `AttributesToJSON` / `JoltTransformJSON`
    - Normalizar al contrato:
    - `weather_event_id`, `warehouse_id`, `temperature_c`, `precipitation_mm`, `wind_kmh`, `weather_code`, `source`, `observation_time`
5. `PublishKafka` (desde `InvokeHTTP` response)
    - Topic raw: `transport.weather.raw`
6. `PublishKafka` (desde JSON normalizado)
    - Topic filtrado: `transport.weather.filtered`
7. `PutFile`
    - Directorio raw local: `/opt/nifi/nifi-current/raw-archive/weather`
8. `raw-hdfs-loader`
    - Sincroniza a HDFS en `/data/raw/nifi/weather`

5.6 Generador GPS para NiFi

El proyecto incluye un generador continuo de logs GPS para alimentar `ListFile/FetchFile`:

- Servicio Docker: `gps-generator`
- Script: `scripts/gps_generator.py`
- Salida: `nifi/input/*.jsonl`

5.7 Bootstrap automatico NiFi

Sin configurar a mano la UI, puedes crear y arrancar el flujo con:

```
./scripts/bootstrap_nifi_flow.sh
```

Este bootstrap crea el Process Group indicado por `NIFI_PG_NAME` (por defecto `kdd_ingestion_auto_v9`) con:

- GPS simulado -> `transport.raw` y `transport.filtered`
- Meteo API publica -> `transport.weather.raw` y `transport.weather.filtered`
- Archivado raw local en `nifi/raw-archive/`

La subida continua a HDFS (`/data/raw/nifi`) la hace el servicio `raw-hdfs-loader`.

6 Dashboard Logistico

6.1 Portada

- Proyecto: `Proyecto Big Data KDD - Logistica`
- Documento: `Especificacion funcional del dashboard`
- Version: `v1.1`
- Fecha: `31/03/2026`

6.2 Indice

1. Objetivo
2. Fuentes de datos del dashboard
3. Que representa cada elemento visual
4. Filtros de Tiempo Real (solo vista izquierda)
5. Filtros de Analisis Logistico (solo vista derecha)
6. Reglas de interpretacion de ruta por vehiculo
7. ETA en panel de vehiculo
8. Datos y nodos actuales
9. Reinicio limpio de demo
10. Nota de uso

6.3 Objetivo

El dashboard muestra dos vistas operativas **desacopladas** (independientes), mas el bloque de metricas globales:

- Operacion en tiempo real de vehiculos (mapa y paneles de flota).
- Analisis de red logistica (mapa de red y tablas de rutas/impacto).

6.4 Captura actual

6.5 Fuentes de datos del dashboard

- Vehiculos en tiempo real: fuente primaria `Cassandra` (`transport.vehicle_latest_state`).
- Fallback de vehiculos: ficheros `nifi/input/gps_*.jsonl` si Cassandra no esta disponible.
- Plan de ruta por vehiculo (origen/destino planificados): `nifi/input/.vehicle_path_state.json`.
- Clima reciente: fuente primaria `Cassandra` (`transport.weather_observations_recent`), cargada por Spark Streaming.
- Fallback de clima: `nifi/raw-archive/weather` (snapshots Open-Meteo) si Cassandra no esta disponible.
- Red logistica: `data/graph/vertices.csv`, `data/graph/edges.csv`, `data/master/warehouses.csv`.

Endpoint de diagnostico de fuentes:

```
curl -s http://localhost:8501/api/debug/sources
```

Devuelve:

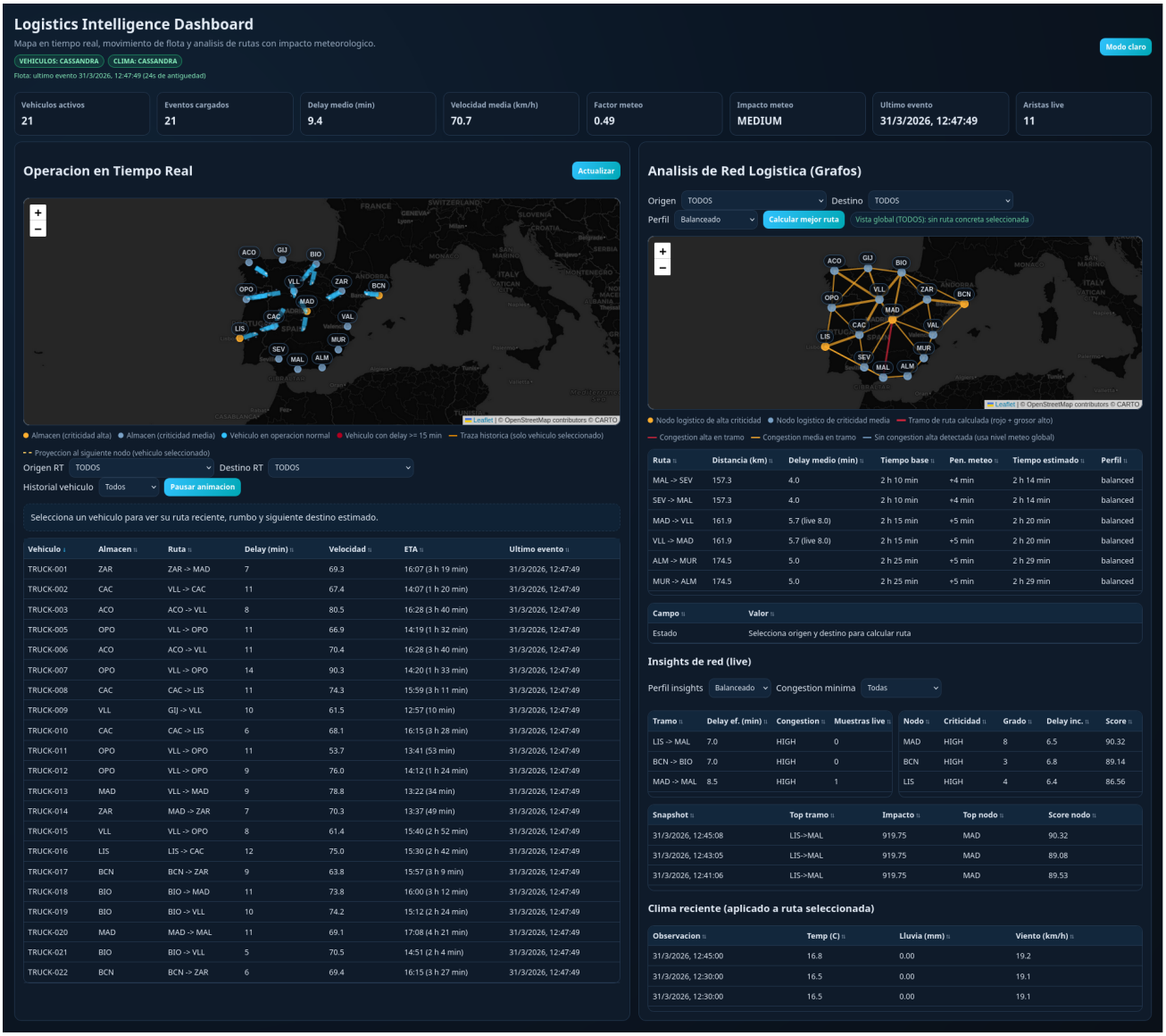

Figure 8: Dashboard logistica (21 vehiculos activos)

- fuente activa para vehiculos (`cassandra` o `nifi_files`),
- estado de conexion a Cassandra y numero de filas obtenidas,
- ultimo fichero GPS detectado en fallback,
- fuente activa de clima (`cassandra` o `nifi_raw_archive`) y estado Cassandra de clima,
- estado/ficheros recientes de clima en fallback,
- conteo de vertices/aristas del grafo.

6.6 Que representa cada elemento visual

6.6.1 Mapa en tiempo real

- Marcadores de vehiculo: posicion actual por `vehicle_id`.
- Lineas naranjas: traza reciente del movimiento del vehiculo (historial corto), **no** son carreteras fijas.
- Linea discontinua amarilla: proyeccion del tramo del vehiculo seleccionado hacia su siguiente nodo estimado.
- Marcadores de almacen:
  - naranja: criticidad `high`
  - azul: criticidad `medium`
  - etiqueta semitransparente para no ocultar vehiculos.
- Etiquetas de nodo mostradas en codigo de 3 letras (ej. `MAD`, `BCN`, `GIJ`), alineadas con la vista de red.

6.6.2 Filtros de Tiempo Real (solo vista izquierda)

- `Origen RT` y `Destino RT` filtran **solo** la vista de Operacion en Tiempo Real.
- Listado de `Origen RT` / `Destino RT` ordenado alfabeticamente.
- `TODOS -> TODOS`: sin filtro (vista global de flota).
- `TODOS -> X`: muestra vehiculos cuya ruta reportada termina en `X`.
- `X -> TODOS`: muestra vehiculos cuya ruta reportada comienza en `X`.
- `X -> Y`: restringe al corredor X-Y con criterio geografico y de ruta.
- Si el vehiculo seleccionado deja de cumplir el filtro, se limpia automaticamente la seleccion.

6.6.3 Mapa de red logistica

- Nodos: almacenes con coordenadas reales (`data/master/warehouses.csv`).
- Aristas: conexiones del grafo (`data/graph/edges.csv`).
- Arista roja: tramo perteneciente a la ruta calculada para el origen/destino seleccionados.
- Color de arista base: severidad meteorologica agregada (low/medium/high).
- Selectores `Origen` / `Destino` ordenados alfabeticamente.
- Delay de arista en tabla/mapa: valor **efectivo** (`effective_avg_delay_minutes`) que mezcla:
  - delay estatico del grafo (`avg_delay_minutes`),
  - telemetria viva de flota (`planned_origin/planned_destination`, delay y velocidad recientes).
- Cuando hay telemetria live en un tramo, se muestra etiqueta `live` en la tabla de rutas.

6.6.4 Tabla de ruta (debajo del mapa logistico)

Muestra:

- Perfil de optimizacion.
- Camino calculado.
- Distancia total.
- Tiempo base sin meteo.
- Penalizacion meteo (en minutos).
- Delay esperado.
- Tiempo estimado final.
- Factor meteorologico aplicado.
- Numero de aristas de la ruta con telemetria live usada en el calculo.
- Nota: el contador `Aristas live` refleja solo tramos de la **ruta seleccionada** con muestras live (`live_sample_count > 0`), no el total del grafo.
- Todas las tablas del dashboard son ordenables por columna (asc/desc) con click en cabecera.

6.6.5 Insights de red (live)

Bloque adicional en la vista de red con dos tablas:

- `Cuellos de botella`: ranking de tramos por `impact_score` (delay efectivo, congestion, distancia y muestras live).
- `Nodos criticos`: ranking de almacenes por `criticality_score` (grado, delay incidente medio, ratio de congestion alta y cobertura live).

Filtros disponibles:

- `Perfil insights`: recalcula score de impacto con los costes del perfil (`balanced`, `fastest`, `resilient`).
- `Congestion minima`: filtra el ranking para mostrar solo tramos/nodos afectados por el nivel indicado (`all`, `low`, `medium`, `high`).
- `Historico insights`: tabla de snapshots recientes persistidos en Cassandra por perfil/congestion.
- Si un tramo llega sin muestras live (`live_sample_count=0`) y congestion `UNKNOWN`, se normaliza a `low/medium/high` con heuristica de delay/distancia para mantener consistencia visual.

Endpoint historico:

```
curl -s "http://localhost:8501/api/network/insights/history?insights_profile=balanced&insights_m
```

6.7 Consolidacion en Hive (reporting)

Se incorpora modo Spark `insights-sync` para consolidar snapshots de Cassandra en Hive:

```
sg docker -c "docker compose exec -T spark-client /opt/spark-app/run-insights-sync.sh"
```

Tablas resultado:

- `transport_analytics.network_insights_snapshots_hive` (detalle de snapshots).
- `transport_analytics.network_insights_hourly_trends` (top tramo y top nodo por hora/perfil/congestion).

6.8 Grafo por cercania geografica (k vecinos)

Para regenerar `data/graph/edges.csv` con una red "casi completa" por cercania:

```
python3 scripts/rebuild_graph_edges_by_proximity.py --k 4
```

Alternativa con variable de entorno:

```
GRAPH_K_NEIGHBORS=5 python3 scripts/rebuild_graph_edges_by_proximity.py
```

Reglas de negocio que aplica el script al regenerar aristas:

- Conexiones directas prohibidas: `BCN-MUR`, `ACO-BIO`, `VAL-ALM`.
- Conexiones obligatorias: `BCN-VAL`, `VAL-MUR`, `ACO-GIJ`, `GIJ-BIO`, `MAD-SEV`, `MAD-MAL`.
- Corredor sur sin atajos directos: `CAC-SEV-MAL-ALM-MUR`.

6.8.1 Filtros de Analisis Logistico (solo vista derecha)

- `Origen` y `Destino` afectan **solo** al bloque de Analisis de Red Logistica.
- `TODOS` en alguno de los selectores pone el bloque en modo vista global (sin ruta concreta).
- No se permite ruta concreta con mismo origen y destino.
- `Calcular mejor ruta` aplica el perfil seleccionado (`balanced`, `fastest`, `resilient`) solo al bloque logistico.

6.9 Reglas de interpretacion de ruta por vehiculo

El dashboard prioriza para cada vehiculo:

1. `planned_origin/planned_destination` (estado del generador en `.vehicle_path_state.json`).
2. Si no existe plan, inferencia por posicion y heading.

Esto evita incoherencias visuales (ejemplo: rutas tipo `X -> X` o proyecciones a nodos no esperados).

6.10 ETA en panel de vehiculo

La ETA (`Hora estimada llegada`) se calcula en frontend por vehiculo como:

1. distancia al siguiente nodo estimado,
2. velocidad actual (`speed_kmh`),
3. buffer parcial de delay (`delay_minutes`).

Para estabilidad visual existe suavizado temporal, pero con resincronizacion forzada en estos casos:

- cambio de destino estimado (`destinationId`),
- divergencia grande entre ETA cacheado y ETA fisico (`distance/speed + delay`).

Con esto se evita que persistan ETAs irreales tras saltos de posicion, cambios de ruta o refrescos incompletos.

6.11 Datos y nodos actuales

Red actual (15 nodos): `ACO, ALM, BCN, BIO, CAC, GIJ, LIS, MAD, MAL, MUR, OPO, SEV, VAL, VLL, ZAR`.

Flota actual (15 vehiculos):

- Activos: `14`
- Mantenimiento: `1` (`TRUCK-004`)

Archivos fuente:

- `data/master/warehouses.csv`
- `data/graph/vertices.csv`
- `data/graph/edges.csv`
- `scripts/gps_generator.py`

6.12 Reinicio limpio de demo

Para empezar de cero y evitar sesgo por historico antiguo:

```
./scripts/reset_demo_data.sh
```

Incluye:

- limpieza de eventos locales en `nifi/input` y raw weather/failures,
- reset de estado streaming (Hive + checkpoints curated),
- recreacion de `gps-generator` y `raw-hdfs-loader`.

Modo agresivo (tambien limpia raw HDFS `/data/raw/nifi`):

```
./scripts/reset_demo_data.sh --hard
```

6.13 Nota de uso

Tras cambios de frontend, hacer recarga fuerte (`Ctrl+F5`) en `http://localhost:8501`.

7 Operaciones del Proyecto KDD

7.1 Portada

- Proyecto: `Proyecto Big Data KDD - Logistica`
- Documento: `Guia de operaciones y troubleshooting`
- Version: `v1.1`
- Fecha: `31/03/2026`

7.2 Indice

1. Arranque y parada
2. Healthcheck en Airflow
3. Arranque del stack desde Airflow
4. Flujo NiFi
5. Validaciones
6. Politica de almacenamiento
7. Matriz operativa end-to-end
8. Limpieza de estado streaming
9. Limpieza de historico failed en Airflow
10. Zona horaria
11. Vistas Hive en hora Madrid
12. Bitacora operativa
13. Operacion del dashboard (estado final)
14. Troubleshooting dashboard
15. Sync de insights Cassandra -> Hive

Este documento resume los comandos recomendados para operar el entorno y validar resultados.

7.3 Arranque y parada

Arrancar todo el stack:

```
./scripts/start_kdd.sh
```

El script de arranque incluye automaticamente:

1. bootstrap de NiFi,
2. `ensure_hive_streaming_compat.sh` (recreacion idempotente de tablas/vistas streaming Hive),
3. sanity check Hive streaming (tablas + vistas Madrid + `COUNT(*)` de ambas vistas),
4. repoblado de `transport_analytics.weather_observations_streaming` desde Cassandra si `v_weather_observations_madrid` queda en `0` filas.

Parar todo el stack:

```
./scripts/stop_kdd.sh
```

7.4 Healthcheck en Airflow

Captura de la vista operativa de Airflow:

Se incorporan dos DAGs de comprobacion:

- `kdd_daily_healthcheck`
- `kdd_hourly_healthcheck`

Estos DAGs validan:

1. Contenedores core activos (`kafka`, `hadoop`, `hive-metastore`, `nifi`, `spark-client`).

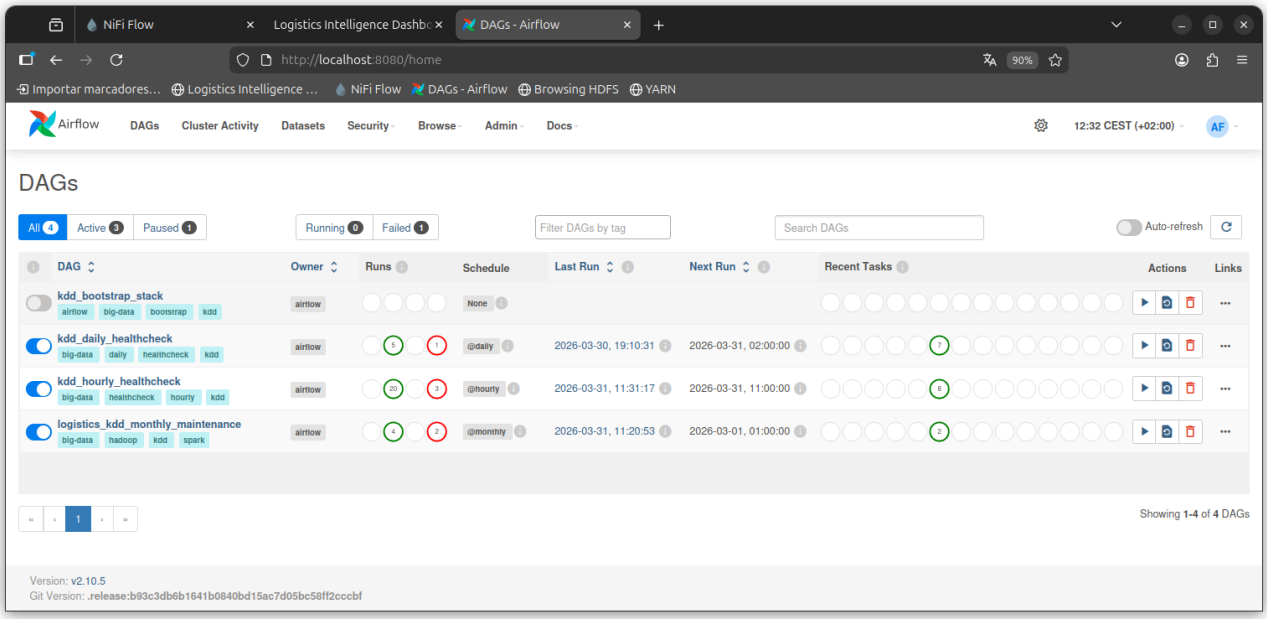

Figure 9: Airflow

2. Topics Kafka necesarios (`transport.filtered`, `transport.weather.filtered`).
3. Paths base de HDFS.
4. Tablas Hive base de `transport_analytics`.
5. Vistas en hora Madrid:
   - `transport_analytics.v_weather_observations_madrid`
   - `transport_analytics.v_delay_metrics_streaming_madrid`
6. Consulta smoke de ambas vistas (con degradacion segura):
   - si las tablas streaming fuente no estan disponibles temporalmente, el smoke no rompe el DAG y valida metastore/batch.
7. Auto-recuperacion meteo:
   - tarea `repopulate_hive_weather_if_empty`,
   - si `weather_rows=0` en `v_weather_observations_madrid`, ejecuta `./scripts/repopulate_hive_weathe` antes del smoke query de meteo.

Alertas:

- Cada DAG incluye `on_failure_callback` (alerta en logs de Airflow).
- Si defines `AIRFLOW_ALERT_EMAIL`, se habilita envio de email en fallo.

7.5 Arranque del stack desde Airflow

DAG de bootstrap:

- `kdd_bootstrap_stack`

Este DAG arranca servicios no-Airflow del stack (`kafka`, `nifi`, `gps-generator`, `hadoop`, `raw-hdfs-loader`, `postgres-metastore`, `hive-metastore`, `cassandra`, `spark-client`) y muestra estado final.

Script recomendado (arranca Airflow y dispara el DAG):

```
./scripts/start_airflow_then_stack.sh
```

7.6 Flujo NiFi

Bootstrap automatico del flujo de ingesta:

```
./scripts/bootstrap_nifi_flow.sh
```

Estructura visual creada por defecto:

- Process Group: `kdd_ingestion_auto_v9`
- Subgrupos:
  - `gps_ingestion`
  - `weather_ingestion`
- Archivado GPS:
  - `SplitText` + `UpdateAttribute` + `PutFile` con filename unico por split (`${filename}_${fragment.index`

Limpieza de grupos legacy (para dejar solo la version actual):

```
./scripts/cleanup_nifi_legacy_pgs.py
```

7.7 Validaciones

Validacion E2E principal (batch + streaming):

```
./scripts/validate_hive_pipeline.sh
```

Validacion especifica del parseo meteo (soporta numeros y numeros como string):

```
./scripts/validate_weather_parsing_fix.sh
```

7.8 Politica de almacenamiento

El proyecto sigue el criterio recomendado:

- Capa `raw`: conservar entrada original en `JSON/JSONL` para trazabilidad y reprocesado.
- Capa `curated`: persistir salida transformada en `Parquet` para analitica eficiente.

Rutas principales:

- Raw en HDFS:
  - `/data/raw/gps_events.jsonl`
  - `/data/raw/nifi/...`
- Curated en HDFS:
  - `/data/curated/enriched_events`
  - `/data/curated/delay_metrics_streaming`
  - `/data/curated/weather_observations_streaming`
  - `/data/curated/enriched_events_streaming`

Comprobacion de rutas y formatos:

```
sg docker -c "docker compose exec -T hadoop hdfs dfs -ls -R /data/raw | head -n 50"
sg docker -c "docker compose exec -T hadoop hdfs dfs -ls -R /data/curated | head -n 100"
```

Comprobacion SQL en Hive:

```
sg docker -c "docker compose exec -T spark-client spark-sql -e \"SHOW TABLES IN transport_analyt
sg docker -c "docker compose exec -T spark-client spark-sql -e \"DESCRIBE FORMATTED transport_an
```

7.9 Consultas desde terminal (Hive y Cassandra)

7.9.1 Hive (via spark-sql en `spark-client`)

Listar tablas del esquema:

```
docker compose exec -T spark-client spark-sql -e "SHOW TABLES IN transport_analytics;"
```

Ver estructura de una tabla:

```
docker compose exec -T spark-client spark-sql -e "DESCRIBE transport_analytics.delay_metrics_str
```

Consultar ultimos registros (primero valida que haya filas):

```
docker compose exec -T spark-client spark-sql -e "SELECT COUNT(*) AS weather_rows FROM transport
docker compose exec -T spark-client spark-sql -e "SELECT weather_event_id, weather_timestamp_utc
```

Si `weather_rows = 0`, la vista existe pero no hay carga streaming meteo en Hive en ese momento.
Como consulta operativa inmediata usa Cassandra:

```
docker compose exec -T cassandra cqlsh -e "SELECT bucket, weather_timestamp, weather_event_id, w
```

Para repoblar Hive meteo desde Cassandra:

```
./scripts/repopulate_hive_weather_from_cassandra.sh
```

Comprobar si existe la tabla streaming fisica (puede no existir y seguir siendo correcto en este proyecto):

```
docker compose exec -T spark-client spark-sql -e "SHOW TABLES IN transport_analytics LIKE 'weath
```

Reparar/asegurar objetos streaming de Hive (tablas + vistas Madrid) tras limpiezas:

```
./scripts/ensure_hive_streaming_compat.sh
```

Sanity check manual (equivalente al que ejecuta `start_kdd.sh`):

```
docker compose exec -T spark-client spark-sql -e "
SHOW TABLES IN transport_analytics LIKE 'delay_metrics_streaming';
SHOW TABLES IN transport_analytics LIKE 'enriched_events_streaming';
SHOW TABLES IN transport_analytics LIKE 'weather_observations_streaming';
SHOW TABLES IN transport_analytics LIKE 'v_delay_metrics_streaming_madrid';
SHOW TABLES IN transport_analytics LIKE 'v_weather_observations_madrid';
SELECT COUNT(*) AS delay_rows FROM transport_analytics.v_delay_metrics_streaming_madrid;
SELECT COUNT(*) AS enriched_stream_rows FROM transport_analytics.enriched_events_streaming;
```

```
SELECT COUNT(*) AS weather_rows FROM transport_analytics.v_weather_observations_madrid;
"
```

7.9.2 Cassandra (via `cqlsh` en contenedor `cassandra`)

Listar keyspaces:

```
docker compose exec -T cassandra cqlsh -e "DESCRIBE KEYSPACES;"
```

Listar tablas del keyspace `transport`:

```
docker compose exec -T cassandra cqlsh -e "DESCRIBE TABLES IN transport;"
```

Ver definicion de tabla:

```
docker compose exec -T cassandra cqlsh -e "DESCRIBE TABLE transport.vehicle_latest_state;"
```

Consultar estado de flota:

```
docker compose exec -T cassandra cqlsh -e "SELECT vehicle_id, warehouse_id, route_id, last_event
```

Consultar clima reciente:

```
docker compose exec -T cassandra cqlsh -e "SELECT weather_timestamp, warehouse_id, temperature_c
```

Consultar snapshots de insights de red:

```
docker compose exec -T cassandra cqlsh -e "SELECT bucket, profile, min_congestion, snapshot_time
```

7.10 Sync de insights Cassandra -> Hive

Para consolidar snapshots de insights en Hive:

```
sg docker -c "docker compose exec -T spark-client /opt/spark-app/run-insights-sync.sh"
```

Tablas resultantes en Hive:

- `transport_analytics.network_insights_snapshots_hive`
- `transport_analytics.network_insights_hourly_trends`

Validacion rapida:

```
docker compose exec -T spark-client spark-sql -e "SELECT COUNT(*) AS c FROM transport_analytics.
docker compose exec -T spark-client spark-sql -e "SELECT hour_bucket, profile, min_congestion, t
```

Si aparece error `permission denied` al ejecutar `run-insights-sync.sh`:

```
chmod +x spark-app/run-insights-sync.sh
sg docker -c "docker compose up -d --build spark-client"
```

7.11 Matriz operativa end-to-end

Tabla de referencia rapida para trazar cada flujo desde origen hasta consumo analitico:

Flujo	Fuente	Kafka	HDFS	Hive
GPS raw	`gps-generator` -> NiFi (`nifi/input/*.jsonl`)	`transport.raw`	`/data/raw/nifi/gps`	No aplica (capa raw)
GPS filtrado/normalizado	NiFi	`transport.filtered`	`/data/curated/delay` (fallback parquet streaming)	transport_analytics.delay_
GPS enriquecido streaming	Spark streaming	`transport.filtered`	`/data/curated/enriche` (fallback parquet streaming)	transport_analytics.enriche
Weather raw	API Open-Meteo -> NiFi	`transport.weather.raw`	`/data/raw/nifi/weather`	No aplica. (capa raw)
Weather filtrado/normalizado	NiFi	`transport.weather.filtered`	`/data/curated/weather` (fallback parquet streaming)	transport_analytics.weather (vista operativa)
Batch historico GPS	HDFS seed/raw (`/data/raw/gps_events.jsonl`)	No aplica	`/data/curated/enriche`	transport_analytics.enriche transport_analytics.delay_ transport_analytics.route_
Grafo shortest path	`data/graph/*` (batch Spark)	No aplica	No aplica (tabla Hive)	transport_analytics.route_
ML riesgo retraso	`enriched_events` (batch Spark ML)	No aplica	`hdfs://hadoop:9000/models/delay_risk_ml` (modelo)	transport_analytics.ml_dela

Checklist de comprobacion rapida:

```
sg docker -c "docker compose exec -T kafka /opt/kafka/bin/kafka-topics.sh --bootstrap-server kaf
sg docker -c "docker compose exec -T hadoop hdfs dfs -ls -R /data/raw/nifi | tail -n 40"
sg docker -c "docker compose exec -T hadoop hdfs dfs -ls -R /data/curated | tail -n 60"
sg docker -c "docker compose exec -T spark-client spark-sql -e \"SHOW TABLES IN transport_analyt
```

7.12 Limpieza de estado streaming

Elimina tablas de streaming en Hive y checkpoints en HDFS para arrancar limpio:

```
./scripts/reset_streaming_state.sh
```

7.13 Limpieza de historico failed en Airflow

Para limpiar runs fallidos antiguos (sin tocar runs `success`):

```
./scripts/cleanup_airflow_failed_runs.sh
```

Aplicar borrado:

```
./scripts/cleanup_airflow_failed_runs.sh --apply
```

Estado esperado tras limpieza:

- Sin `failed` para:
  - `kdd_daily_healthcheck`
  - `kdd_hourly_healthcheck`

Comprobacion SQL directa en metastore de Airflow:

```
docker exec airflow-postgres psql -U airflow -d airflow -c "
SELECT dag_id, count(*) AS failed_runs
FROM dag_run
WHERE state='failed'
  AND dag_id IN ('kdd_daily_healthcheck','kdd_hourly_healthcheck')
GROUP BY dag_id;"
```

7.14 Zona horaria

- Spark usa por defecto `SPARK_SQL_TIMEZONE=Europe/Madrid`.
- Los eventos entran habitualmente con marca temporal UTC (`...Z`), por lo que puede ser util consultar tanto UTC como Madrid.

7.15 Vistas Hive en hora Madrid

Vistas disponibles:

- `transport_analytics.v_weather_observations_madrid`
- `transport_analytics.v_delay_metrics_streaming_madrid`

Si una vista falla con `TABLE_OR_VIEW_NOT_FOUND`, ejecutar:

```
./scripts/ensure_hive_streaming_compat.sh
```

Consulta ejemplo meteo:

```
SELECT weather_event_id, weather_timestamp_utc, weather_timestamp_madrid
FROM transport_analytics.v_weather_observations_madrid
ORDER BY weather_timestamp_utc DESC
LIMIT 10;
```

Consulta ejemplo GPS agregado:

```
SELECT warehouse_id, window_start_utc, window_start_madrid, window_end_utc, window_end_madrid
FROM transport_analytics.v_delay_metrics_streaming_madrid
ORDER BY window_start_utc DESC
LIMIT 10;
```

7.16 Bitacora operativa (actualizada 31/03/2026)

Cambios aplicados:

1. Healthchecks Airflow estabilizados frente a ausencia temporal de tablas streaming fuente.

2. Historial rojo de healthchecks limpiado en Airflow.
3. NiFi reorganizado por dominios (`gps_ingestion` y `weather_ingestion`) en `kdd_ingestion_auto_v9`.
4. Process Group legacy eliminado para evitar ruido visual en la UI de NiFi.
5. Incorporado flujo de insights de red con persistencia en Cassandra y sync a Hive.

7.17 Nota de tablas en tiempo real

- `transport_analytics.enriched_events` es historico batch.
- Para consultas en vivo de eventos enriquecidos usar `transport_analytics.enriched_events_streaming`.

7.18 Operacion del dashboard (estado final)

7.18.1 Desacople de vistas

El dashboard opera con dos contextos de filtro independientes:

1. `Operacion en Tiempo Real` (izquierda):
   - filtros: `Origen RT`, `Destino RT`.
2. `Analisis de Red Logistica` (derecha):
   - filtros: `Origen`, `Destino`, `Perfil`.

Un cambio de filtro en un bloque no debe alterar el otro.

Adicionalmente, en el estado actual:

1. Los selectores de origen/destino en ambas vistas se cargan ordenados alfabeticamente.
2. Todas las tablas del dashboard admiten ordenacion por columna.

7.18.2 Reglas de filtro en Tiempo Real

- `TODOS -> TODOS`: muestra toda la flota.
- `TODOS -> X`: muestra vehiculos cuyo destino reportado es `X`.
- `X -> TODOS`: muestra vehiculos cuyo origen reportado es `X`.
- `X -> Y`: restringe por corredor y coherencia espacial.

7.18.3 Fuente de ruta por vehiculo

Para evitar inconsistencias de inferencia:

1. Se usa plan de trayecto por vehiculo desde `nifi/input/.vehicle_path_state.json`.
2. Ese plan se expone por API en cada item como:
   - `planned_origin`
   - `planned_destination`
3. El frontend prioriza estos campos para `Ruta reportada`, `Siguiente nodo` y linea de proyeccion.

7.19 Troubleshooting dashboard

7.19.1 Caso: veo ruta incoherente (`X -> X`) o destino inesperado

1. Forzar recarga de navegador:

```
# navegador
Ctrl+F5
```

2. Comprobar payload de vehiculos:

```
docker compose exec -T dashboard python -c "import json,urllib.request;d=json.load(urllib.reques
```

3. Verificar estado del plan del generador:

```
python3 - <<'PY'
import json
with open('nifi/input/.vehicle_path_state.json','r',encoding='utf-8') as f:
    d=json.load(f)
print(d.get('V1'))
PY
```

Si `planned_origin/planned_destination` no aparecen en API, reiniciar dashboard:

```
docker compose restart dashboard
```

7.19.2 Caso: ETA incoherente (ejemplo: >300 km y ETA en pocos minutos)

Comprobar primero distancia y velocidad mostradas en el panel de vehiculo. Si son coherentes entre si pero la ETA no lo es:

1. Forzar recarga fuerte del frontend (`Ctrl+F5`) y pulsar `Actualizar` en el dashboard.
2. Verificar que el frontend ya incluye el ajuste de resincronizacion de ETA:

```
rg -n "forceResync|divergence|destinationChanged" dashboard/static/app.js
```

3. Reiniciar el servicio dashboard para limpiar estado en memoria:

```
docker compose restart dashboard
```

Nota tecnica: - El calculo ETA ahora fuerza resincronizacion inmediata cuando hay cambio de destino estimado o desviacion grande entre ETA cacheado y ETA fisico (`distance/speed + delay`), evitando ETAs irreales persistentes.

8 Arquitectura del Proyecto

8.1 Portada

- Proyecto: `Proyecto Big Data KDD - Logistica`
- Documento: `Arquitectura tecnica actual`
- Version: `v1.1`
- Fecha: `31/03/2026`

8.2 Indice

1. Vista general

2. Diagrama de arquitectura
3. Componentes y responsabilidades
4. Flujo de datos end-to-end
5. Capas de almacenamiento
6. Decision de desacople en dashboard

8.3 1. Vista general

La plataforma implementa el ciclo KDD completo en local con Docker Compose:

1. Ingesta dual (GPS + clima) en NiFi.
2. Bus de eventos en Kafka (raw y filtered).
3. Persistencia raw en HDFS (sincronizacion desde raw-archive NiFi).
4. Procesamiento batch y streaming con Spark.
5. Explotacion analitica en Hive + estado operativo en Cassandra.
6. Orquestacion y healthchecks con Airflow.
7. Dashboard web con vistas de Tiempo Real y Red Logistica desacopladas.

8.4 2. Diagrama de arquitectura

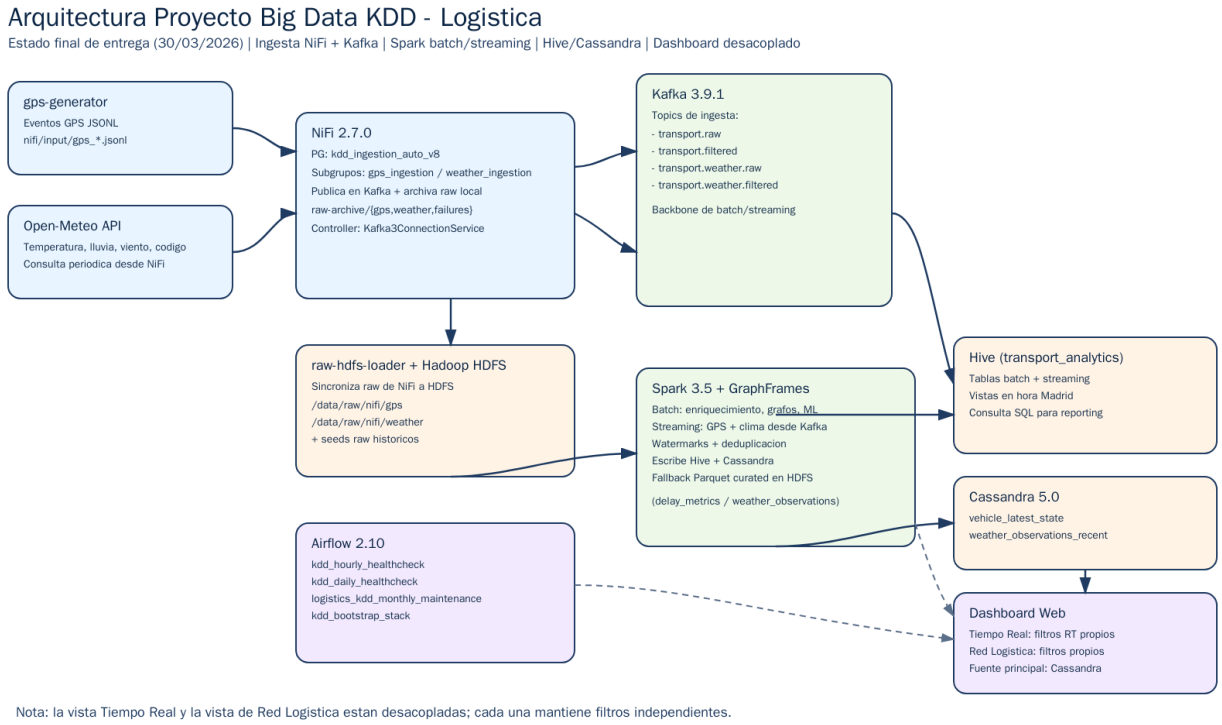

Figure 10: Diagrama de arquitectura

Archivo de imagen:

- `docs/architecture-diagram.png`

8.5 3. Componentes y responsabilidades

- `gps-generator`
  - Genera eventos GPS en `nifi/input/gps_*.jsonl`.
  - Mantiene estado de trayectos en `nifi/input/.vehicle_path_state.json`.
  - Usa rutas realistas derivadas del grafo logistico y flota de `data/master/vehicles.csv`.
- `NiFi 2.7.0`
  - PG principal: `kdd_ingestion_auto_v9`.
  - Subgrupos: `gps_ingestion`, `weather_ingestion`.
  - Publica eventos a Kafka y archiva raw local (`raw-archive`).
  - Evita sobrescrituras de GPS raw con renombrado unico por split (`filename + fragment.index + UUID`).
- `Kafka 3.9.1`
  - Topics:
    * `transport.raw`
    * `transport.filtered`
    * `transport.weather.raw`
    * `transport.weather.filtered`
- `raw-hdfs-loader` + `Hadoop HDFS`
  - Sincronizan raw local de NiFi a:
    * `/data/raw/nifi/gps`
    * `/data/raw/nifi/weather`
- `Spark 3.5 + GraphFrames`
  - Batch: enriquecimiento, agregaciones, grafos, shortest paths y ML.
  - Streaming: GPS y clima desde Kafka con watermark + deduplicacion.
  - Persistencia en Hive y Cassandra (con fallback Parquet curated), incluyendo `transport_analytics.enric`
  - Modo `insights-sync` para consolidar snapshots de insights de red en Hive.
- `Hive`
  - Almacen analitico SQL (`transport_analytics`).
  - Tablas batch + streaming y vistas en hora Madrid.
- `Cassandra`
  - Estado de baja latencia:
    * `transport.vehicle_latest_state`
    * `transport.weather_observations_recent`
    * `transport.network_insights_snapshots`
- `Airflow`
  - `kdd_hourly_healthcheck`
  - `kdd_daily_healthcheck`
  - `kdd_bootstrap_stack`
  - `logistics_kdd_monthly_maintenance`
  - El bootstrap ejecuta `scripts/ensure_hive_streaming_compat.sh` para asegurar vistas/tablas streaming.
- `Dashboard`

  - Fuente principal: Cassandra.
  - Fallback controlado a ficheros NiFi para vehiculos/clima.
  - Filtros desacoplados por vista.
  - Insights live de red y resumen de aristas con telemetria (`live_edge_summary`).

8.6 4. Flujo de datos end-to-end

1. `gps-generator` y `Open-Meteo` alimentan NiFi.
2. NiFi publica a Kafka (raw y filtered) y deja copia raw local.
3. `raw-hdfs-loader` mueve raw a HDFS.
4. Spark streaming consume filtered y escribe Hive/Cassandra.
5. Spark batch recalcula historicos, grafos y modelo ML.
6. Dashboard consulta principalmente Cassandra para operacion.

8.7 5. Capas de almacenamiento

- `Raw (Bronze)`
  - JSON/JSONL original sin transformar.
  - Ejemplo: `/data/raw/nifi/...`
- `Curated (Silver/Gold)`
  - Parquet + tablas Hive para consulta eficiente.
  - Ejemplo: `/data/curated/...` y `transport_analytics.*`

8.8 6. Decision de desacople en dashboard

Se adopta desacople intencional para reducir ambiguedad operativa:

- `Operacion en Tiempo Real` usa filtros `Origen RT` / `Destino RT`.
- `Analisis de Red Logistica` usa `Origen` / `Destino` / `Perfil`.

Un cambio en una vista no altera la otra.

9 Release Notes - 30/03/2026

9.1 Portada

- Proyecto: `Proyecto Big Data KDD - Logistica`
- Documento: `Release notes iteracion 30/03/2026`
- Version: `v1.0-entrega`
- Fecha: `30/03/2026`

9.2 Indice

1. Resumen ejecutivo
2. Dashboard web
3. Datos de red y simulacion

4. Airflow
5. NiFi
6. Scripts nuevos/actualizados
7. Verificaciones recomendadas
8. Impacto operativo

Este documento consolida los cambios funcionales y operativos aplicados durante la iteracion del 30 de marzo de 2026.

9.3 Addendum - 31/03/2026

Cambios adicionales aplicados tras la iteracion base:

9.3.1 Dashboard

- Etiquetas de nodos en Tiempo Real normalizadas a codigos de 3 letras (mismo criterio que Red Logistica).
- Etiquetas de nodo en Tiempo Real con transparencia para no ocultar vehiculos.
- Selectores de origen/destino (RT y red) ordenados alfabeticamente.
- Todas las tablas del dashboard ahora son ordenables por columnas.
- Tabla de datos bajo el mapa logistico ajustada para reflejar origen/destino seleccionados.
- Leyendas refinadas en mapas de Tiempo Real y Red Logistica.

9.3.2 Insights de red

- Nuevo bloque `Insights de red (live)` con:
  - ranking de cuellos de botella,
  - ranking de nodos criticos,
  - historico de snapshots.
- Persistencia de snapshots en Cassandra (`transport.network_insights_snapshots`).
- Endpoint historico: `/api/network/insights/history`.
- Normalizacion de congestion `UNKNOWN` a niveles operativos (`low/medium/high`) cuando faltan muestras live.

9.3.3 Spark / Hive

- Nuevo launcher `spark-app/run-insights-sync.sh` para ejecutar modo `insights-sync`.
- Consolidacion Cassandra -> Hive:
  - `transport_analytics.network_insights_snapshots_hive`
  - `transport_analytics.network_insights_hourly_trends`

9.3.4 Red y simulacion

- Red actual consolidada en 15 nodos:
  - `ACO, ALM, BCN, BIO, CAC, GIJ, LIS, MAD, MAL, MUR, OPO, SEV, VAL, VLL, ZAR`
- Flota actual consolidada en 15 vehiculos (`14` activos + `1` mantenimiento).

- Regeneracion de aristas por proximidad con `scripts/rebuild_graph_edges_by_proximity.py`.
- Reglas de negocio en la topologia:
  - prohibidas: `BCN-MUR`, `ACO-BIO`, `VAL-ALM`,
  - obligatorias: `BCN-VAL`, `VAL-MUR`, `ACO-GIJ`, `GIJ-BIO`, `MAD-SEV`, `MAD-MAL`,
  - corredor sur sin atajos: `CAC-SEV-MAL-ALM-MUR`.

9.4 Resumen ejecutivo

Se estabilizo la plataforma end-to-end para demo operativa en logistica:

- Dashboard mas legible y accionable (mapa tiempo real + analisis de red).
- Healthchecks de Airflow robustos y sin historico rojo pendiente.
- NiFi reestructurado visualmente por dominios (`GPS` y `Clima`).
- NiFi actualizado a `kdd_ingestion_auto_v9` con archivado GPS sin sobrescrituras.
- Scripts de arranque/limpieza alineados para reproducibilidad.
- Spark streaming ampliado con tabla de eventos enriquecidos en tiempo real (`transport_analytics.enriched_`
- Generador GPS actualizado a rutas realistas sobre grafo logistico y flota actual (`TRUCK-*`).

9.5 Dashboard web

Cambios relevantes incorporados:

- Modo oscuro por defecto y soporte de cambio claro/oscuro.
- Mapa en tiempo real simplificado para reducir ruido visual.
- Seleccion de vehiculo con foco de ruta y panel de detalles.
- Ajustes de trazas para reducir "teletransportaciones" percibidas.
- Mejora de marcadores de vehiculo y orientacion en sentido de marcha.
- Inclusiones de ETA en panel de vehiculo.
- Correccion de ETA incoherente por caché/suavizado:
  - resincronizacion inmediata cuando cambia el destino estimado,
  - resincronizacion inmediata ante divergencia grande entre ETA cacheado y ETA fisico (`distancia/velocidad + delay`).
- Tabla de rutas en analisis logistico con tiempos en formato horas/minutos.
- Etiquetas de ciudad y mejoras de legibilidad en el mapa de red.
- Sustitucion de paneles/graficos redundantes por tablas de datos de rutas.
- Calculo de clima y factor meteorologico enlazado a ruta seleccionada.
- Desacople de filtros entre vistas:
  - filtros propios para Tiempo Real (`Origen RT`, `Destino RT`),
  - filtros propios para Red Logistica (`Origen`, `Destino`, `Perfil`).
- Modo `TODOS` soportado en ambos contextos de filtro.
- Regla de no permitir origen=destino en ruta concreta de Red Logistica.
- Actualizacion inmediata de la vista de Tiempo Real al cambiar sus filtros.
- Correcciones de coherencia de ruta por vehiculo:
  - eliminacion de casos `X -> X` no deseados,

  - eliminacion de proyecciones a nodos no esperados (ej. Lisboa cuando la ruta reportada indica otro destino).
- Enriquecimiento backend de `/api/vehicles/latest` y `/api/vehicles/history` con:
  - `planned_origin`,
  - `planned_destination`, tomados de `nifi/input/.vehicle_path_state.json`.

9.6 Datos de red y simulacion

- Actualizacion de nodos/ciudades de la red.
- Inclusiones y ajustes de almacenes para mantener consistencia entre:
  - mapa en tiempo real,
  - mapa de red logistica,
  - calculo de rutas.

9.7 Airflow

9.7.1 DAGs impactados

- `kdd_hourly_healthcheck`
- `kdd_daily_healthcheck`

9.7.2 Problema

Las tareas smoke contra vistas Madrid podian fallar cuando no existian temporalmente las tablas streaming fuente:

- `transport_analytics.weather_observations_streaming`
- `transport_analytics.delay_metrics_streaming`

Error observado: `TABLE_OR_VIEW_NOT_FOUND`.

9.7.3 Solucion aplicada

Se ajustaron `smoke_query_weather` y `smoke_query_delay` para:

1. intentar consulta normal sobre vistas Madrid,
2. degradar a validacion segura (metastore/batch) si la fuente streaming no esta disponible.
3. mantener como referencia operativa meteo la vista `transport_analytics.v_weather_observations_madrid` (en lugar de depender de la tabla `weather_observations_streaming`).

Se incorpora ademas auto-recuperacion meteo en ambos DAGs (`kdd_hourly_healthcheck` y `kdd_daily_healthcheck`):

4. nueva tarea `repopulate_hive_weather_if_empty`,
5. si `weather_rows=0` en `v_weather_observations_madrid`, ejecuta `./scripts/repopulate_hive_weather_fro`
6. despues ejecuta el smoke query meteo para validar que la vista vuelve a ser consultable.

Resultado:

- ejecuciones manuales validadas en `success`,
- limpieza de runs `failed` historicos de healthchecks,
- sin rojos pendientes para esos DAGs.
- se incorpora `scripts/ensure_hive_streaming_compat.sh` para recrear tablas/vistas streaming de Hive tras limpiezas y evitar `TABLE_OR_VIEW_NOT_FOUND`.

9.8 NiFi

9.8.1 Bootstrap y visualizacion

Se mejoro el bootstrap para crear una estructura separada por dominio:

- PG principal: `kdd_ingestion_auto_v9`
- Subgrupos:
  - `gps_ingestion`
  - `weather_ingestion`

Objetivo:

- simplificar navegacion en UI,
- evitar canvas unico sobrecargado,
- facilitar diagnostico por flujo.

9.8.2 Limpieza de legacy

Se elimino el Process Group legacy `kdd_ingestion_auto` tras:

- stop de componentes,
- vaciado de colas,
- deshabilitado de controller services,
- borrado final por API.

9.9 Scripts nuevos/actualizados

9.9.1 Nuevos

- `scripts/cleanup_nifi_legacy_pgs.py`
  - Limpia PGs legacy por prefijo.
  - Mantiene un PG objetivo (por defecto `kdd_ingestion_auto_v9`).
  - Gestiona colas, controller services y borrado con reintentos.
- `scripts/repopulate_hive_weather_from_cassandra.sh`
  - Exporta snapshots meteo desde Cassandra (`transport.weather_observations_recent`).
  - Repuebla `transport_analytics.weather_observations_streaming` en Hive.
  - Verifica recuento final de `transport_analytics.v_weather_observations_madrid`.

9.9.2 Actualizados

- `scripts/bootstrap_nifi_flow.py`

  - crea subgrupos `gps_ingestion` y `weather_ingestion`,
  - incorpora renombrado unico por split para archivado GPS (`${filename}_${fragment.index}_${UUID()}`
  - mantiene wiring y controller service Kafka.
- `scripts/gps_generator.py`
  - genera rutas realistas desde el grafo de ciudades (ya no etiquetas aleatorias fijas),
  - carga flota desde `data/master/vehicles.csv`,
  - permite reasignacion dinamica de ruta al cerrar tramos.
- `spark-app/src/main/java/com/proyectobigdata/LogisticsAnalyticsJob.java`
  - parseo robusto de timestamps ISO UTC para GPS y clima,
  - nueva salida Hive `transport_analytics.enriched_events_streaming` en modo streaming.
- `scripts/bootstrap_nifi_flow.sh`
  - `NIFI_PG_NAME` por defecto: `kdd_ingestion_auto_v9`.
- `scripts/start_kdd.sh`
  - exporta `NIFI_PG_NAME` por defecto a `kdd_ingestion_auto_v9` antes de bootstrap.
  - si `v_weather_observations_madrid` queda vacia, dispara repoblado desde Cassandra.

9.10 Verificaciones recomendadas

9.10.1 Airflow

```
docker exec airflow-postgres psql -U airflow -d airflow -c "
SELECT dag_id, count(*) AS failed_runs
FROM dag_run
WHERE state='failed'
  AND dag_id IN ('kdd_daily_healthcheck','kdd_hourly_healthcheck')
GROUP BY dag_id;"
```

9.10.2 NiFi

Comprobar en UI/API que en root solo queda el PG activo esperado y sus subgrupos:

- `kdd_ingestion_auto_v9`
- `gps_ingestion`
- `weather_ingestion`

9.11 Impacto operativo

- Menos incidencias visuales y de UX en demo de dashboard.
- Menos falsos negativos en healthchecks por dependencia de estado streaming.
- Arranque mas predecible del flujo NiFi.
- Menor deuda operativa al eliminar estructura legacy.

10 Proyecto Big Data KDD - Java 21 + Docker

Implementacion de una plataforma Big Data orientada a logistica y transporte siguiendo el ciclo KDD del enunciado, usando Java 21 para la aplicacion Spark y una arquitectura desplegada con contenedores

Docker.

10.1 Arquitectura

- `NiFi 2.7.0`: ingesta desde API/logs simulados y publicacion en Kafka.
- `Kafka 3.9.1 (KRaft)`: bus de eventos para crudo y filtrado.
- `gps-generator`: generador continuo de logs GPS para alimentar NiFi (`nifi/input`).
- `raw-hdfs-loader`: sincronizacion continua de raw archivado por NiFi hacia HDFS (`/data/raw/nifi`).
- `Hadoop 3.4.2 pseudodistribuido`: HDFS + YARN en un unico nodo.
- `Hive`: almacenamiento SQL historico y catalogo.
- `Cassandra 5.0`: ultimo estado por vehiculo para consultas de baja latencia.
- `Spark 3.5.x + GraphFrames`: limpieza, enriquecimiento, streaming, analisis de grafos y modelo ML de riesgo de retraso.
- `Airflow 2.10.x`: orquestacion del mantenimiento mensual.
- `Dashboard Web`: visualizacion en tiempo real de vehiculos (mapa), trazas de movimiento y analitica de rutas con grafo + impacto meteorologico.

10.2 Estructura

- `docker-compose.yml`: stack completo.
- `docker/`: Dockerfiles y configuracion de infraestructura.
- `spark-app/`: proyecto Maven en Java 21 para Spark.
- `dags/`: DAGs de Airflow.
- `data/`: datasets semilla para demo.
- `nifi/`: material para la ingesta y notas del flujo.
- `docs/`: documentacion tecnica y pasos operativos.
- `dashboard/`: servidor web ligero + frontend del dashboard logistico.

Indice maestro de entrega:

- docs/ENTREGA.md
- docs/architecture.md (incluye diagrama docs/architecture-diagram.png)
- CHANGELOG.md (historial consolidado de cambios)

10.3 Arranque rapido

1. Construir y levantar la plataforma:

   ```
   docker compose up --build -d
   ```

2. Compilar la aplicacion Spark:

   ```
   docker compose exec spark-client mvn -f /opt/spark-app/pom.xml -DskipTests package
   ```

3. Cargar datasets de demo en HDFS:

   ```
   docker compose exec hadoop bash /opt/hadoop/bootstrap/load-seed-data.sh
   ```

4. Ejecutar el job batch/grafo:

   ```
   docker compose exec spark-client bash /opt/spark-app/run-batch.sh
   ```

   Opcional (sincronizar snapshots de insights Cassandra -> Hive):

   ```
   sg docker -c "docker compose exec -T spark-client /opt/spark-app/run-insights-sync.sh"
   ```

5. Ejecutar el job de streaming:

   ```
   docker compose exec spark-client bash /opt/spark-app/run-streaming.sh
   ```

   Opcional (forzar reproceso desde el inicio de los topics):

   ```
   docker compose exec -e STREAMING_STARTING_OFFSETS=earliest spark-client bash /opt/spark-app/
   ```

   Opcional (zona horaria SQL de Spark; por defecto `Europe/Madrid`):

   ```
   docker compose exec -e SPARK_SQL_TIMEZONE=Europe/Madrid spark-client bash /opt/spark-app/ru
   ```

6. Configurar y arrancar flujo NiFi automaticamente (sin UI manual):

   ```
   ./scripts/bootstrap_nifi_flow.sh
   ```

   El bootstrap usa por defecto `NIFI_PG_NAME=kdd_ingestion_auto_v9` y crea dos subgrupos:

   - `gps_ingestion`
   - `weather_ingestion`

7. Abrir dashboard logistico (mapa + grafos + meteo):

   ```
   docker compose up -d dashboard
   ```

   URL:

   - `http://localhost:8501`

   Comportamiento actual de filtros:

   - Vista Tiempo Real (izquierda): usa `Origen RT` / `Destino RT` y afecta solo esa vista.
   - Vista Analisis Logistico (derecha): usa `Origen` / `Destino` y afecta solo mapa/logistica/tablas de esa columna.
   - Ambas vistas estan desacopladas intencionalmente.

10.4 Scripts operativos

- Arranque completo del stack:

  ```
  ./scripts/start_kdd.sh
  ```

  Incluye automaticamente:

  - bootstrap de flujo NiFi,
  - `scripts/ensure_hive_streaming_compat.sh`,
  - sanity check Hive (`SHOW TABLES` + `COUNT(*)` sobre vistas Madrid),
  - repoblado automatico de meteo en Hive desde Cassandra si `weather_rows=0`.

- Parada completa del stack:

  ```
  ./scripts/stop_kdd.sh
  ```

- Limpieza de estado streaming (tablas Hive + checkpoints HDFS):

  ```
  ./scripts/reset_streaming_state.sh
  ```

- Reset de demo (limpieza de eventos locales + reset streaming + recrear generadores):

  ```
  ./scripts/reset_demo_data.sh
  ```

  Hard reset (incluye limpieza de `/data/raw/nifi` en HDFS):

  ```
  ./scripts/reset_demo_data.sh --hard
  ```

10.5 Validacion automatica Hive

Para validar en un solo comando la escritura batch + streaming en Hive:

```
chmod +x scripts/validate_hive_pipeline.sh
./scripts/validate_hive_pipeline.sh
```

Documentacion detallada:

- `docs/hive-validation.md`

Para limpiar tablas/checkpoints de streaming (quitar estado de pruebas):

```
chmod +x scripts/reset_streaming_state.sh
./scripts/reset_streaming_state.sh
```

Validacion especifica del fix de parseo meteo (numeros y numeros-string):

```
./scripts/validate_weather_parsing_fix.sh
```

Garantizar compatibilidad Hive de objetos streaming (tablas/vistas Madrid):

```
./scripts/ensure_hive_streaming_compat.sh
```

Repoblar Hive meteo desde Cassandra (cuando `v_weather_observations_madrid` esta vacia):

```
./scripts/repopulate_hive_weather_from_cassandra.sh
```

10.6 Formato de datos (raw vs procesado)

Se aplica la separacion recomendada:

- `Raw`: eventos sin transformar en `JSON/JSONL`.
- `Procesado`: datasets limpios y agregados en `Parquet` (consultables via Hive).

Referencias del proyecto:

- Raw GPS base Spark batch: `hdfs://hadoop:9000/data/raw/gps_events.jsonl`
- Raw NiFi archivado en host: `nifi/raw-archive/`
- Raw NiFi sincronizado a HDFS: `/data/raw/nifi`

- Curated batch en Parquet: `hdfs://hadoop:9000/data/curated/enriched_events`
- Curated streaming fallback Parquet:
  - `hdfs://hadoop:9000/data/curated/delay_metrics_streaming`
  - `hdfs://hadoop:9000/data/curated/weather_observations_streaming`
  - `hdfs://hadoop:9000/data/curated/enriched_events_streaming`

Comprobacion rapida:

```
sg docker -c "docker compose exec -T hadoop hdfs dfs -ls -R /data/raw | head -n 40"
sg docker -c "docker compose exec -T hadoop hdfs dfs -ls -R /data/curated | head -n 80"
```

10.6.1 Matriz operativa (resumen)

Flujo	Kafka	HDFS	Hive
GPS raw	`transport.raw`	`/data/raw/nifi/gps/...`	No aplica
GPS filtrado	`transport.filtered`	`/data/curated/delay_metrics_streaming`	`transport_analytics.delay_metri`
GPS enriquecido streaming	`transport.filtered`	`/data/curated/enriched_events_streaming`	`transport_analytics.enriched_ev`
Weather raw	`transport.weather.raw`	`/data/raw/nifi/weather`	No aplica
Weather filtrado	`transport.weather.filtered`	`/data/curated/weather_observations_streaming`	`transport_analytics.weather_o` (vista operativa)

Batch historico GPS	No aplica	`/data/curated/enriched_events`	`transport_analytics.enriched_ev`
			`transport_analytics.delay_metri`
			`transport_analytics.route_graph`

Detalle operativo completo:

- `docs/operations.md` (seccion `Matriz operativa end-to-end`)

10.7 Vistas Hive en hora Madrid

Se han creado vistas para consultar timestamps en `Europe/Madrid` sin conversion manual:

- `transport_analytics.v_weather_observations_madrid`
- `transport_analytics.v_delay_metrics_streaming_madrid`

Ejemplos:

```sql
SELECT weather_event_id, weather_timestamp_utc, weather_timestamp_madrid
FROM transport_analytics.v_weather_observations_madrid
ORDER BY weather_timestamp_utc DESC
LIMIT 10;
```

```sql
SELECT warehouse_id, window_start_utc, window_start_madrid, window_end_utc, window_end_madrid
FROM transport_analytics.v_delay_metrics_streaming_madrid
ORDER BY window_start_utc DESC
LIMIT 10;
```

Nota de compatibilidad:

- Si el estado streaming fue limpiado, el proyecto recrea objetos base con `scripts/ensure_hive_streaming_comp`
- Esto evita errores `TABLE_OR_VIEW_NOT_FOUND` en las vistas Madrid aunque no haya datos todavia.

10.8 Servicios

- NiFi: `http://localhost:8443/nifi`
- Kafka broker interno: `kafka:9092`
- Hadoop/YARN: `http://localhost:9870`, `http://localhost:8088`
- HiveServer2: `localhost:10000`
- Cassandra CQL: `localhost:9042`
- Airflow: `http://localhost:8080`
- Dashboard logistico: `http://localhost:8501`

Usuario NiFi:

- usuario: `admin`
- password: `adminadminadmin`

Usuario por defecto Airflow:

- usuario: `airflow`
- password: `airflow`

DAGs relevantes:

- `logistics_kdd_monthly_maintenance` (mantenimiento mensual)
- `kdd_daily_healthcheck` (checklist diario automatizado)
- `kdd_hourly_healthcheck` (checklist horario automatizado)
- `kdd_bootstrap_stack` (arranque del stack no-Airflow desde Airflow)

Auto-recuperacion meteo en DAGs de healthcheck:

- `kdd_daily_healthcheck` y `kdd_hourly_healthcheck` incluyen la tarea `repopulate_hive_weather_if_empty`
- Si `SELECT COUNT(*) FROM transport_analytics.v_weather_observations_madrid` devuelve `0`, ejecutan `./scripts/repopulate_hive_weather_from_cassandra.sh` antes del smoke query meteo.
- Si hay filas, no repueblan.

Alertas Airflow:

- Todos los DAGs incorporan callback de fallo.
- Para email en fallo: exporta `AIRFLOW_ALERT_EMAIL` en el entorno de Airflow.

Arranque "Airflow primero y despues stack desde DAG":

```
./scripts/start_airflow_then_stack.sh
```

Limpieza de runs `failed` historicos de healthchecks (modo simulacion + apply):

```
./scripts/cleanup_airflow_failed_runs.sh
./scripts/cleanup_airflow_failed_runs.sh --apply
```

Limpieza de Process Groups legacy de NiFi (manteniendo `kdd_ingestion_auto_v9`):

```
./scripts/cleanup_nifi_legacy_pgs.py
```

10.9 Cambios recientes (31/03/2026)

- Dashboard:
  - Etiquetas de nodos en mapa de Tiempo Real normalizadas a codigo de 3 letras (igual que el mapa logistico).
  - Etiquetas de nodos de Tiempo Real con transparencia para no tapar vehiculos.
  - Selectores `Origen/Destino` (RT y Red Logistica) ordenados alfabeticamente.
  - Todas las tablas del dashboard ordenables por columna (click en cabeceras).
  - Tabla bajo mapa logistico enfocada en origen/destino seleccionados (o global en `TODOS`).
  - Nuevos insights live de red (cuellos de botella, nodos criticos e historico).
  - Resolucion de congestion `UNKNOWN` a `low/medium/high` cuando no hay muestras live.
- Backend dashboard / Spark:
  - Nuevos campos live por arista (`effective_avg_delay_minutes`, `live_avg_delay_minutes`, `live_sample_count`, `congestion_level`) y resumen `live_edge_summary`.
  - Persistencia de snapshots de insights en Cassandra (`transport.network_insights_snapshots`).
  - Endpoint historico de insights: `/api/network/insights/history`.
  - Nuevo modo Spark `insights-sync` + launcher `spark-app/run-insights-sync.sh` para consolidar en Hive:
    * `transport_analytics.network_insights_snapshots_hive`
    * `transport_analytics.network_insights_hourly_trends`
- Red y simulacion:
  - Red actual ampliada a 15 nodos y flota de 15 vehiculos (14 activos + 1 en mantenimiento).
  - Script de regeneracion de aristas por cercania geografica:
    * `python3 scripts/rebuild_graph_edges_by_proximity.py --k 4`
  - Reglas de negocio aplicadas en la red:
    * prohibidas directas: `BCN-MUR`, `ACO-BIO`, `VAL-ALM`
    * obligatorias: `BCN-VAL`, `VAL-MUR`, `ACO-GIJ`, `GIJ-BIO`, `MAD-SEV`, `MAD-MAL`
    * corredor sur sin atajos: `CAC-SEV-MAL-ALM-MUR`
- Operativa:
  - Si aparece `permission denied` al lanzar `run-insights-sync.sh`, ejecutar:
    * `chmod +x spark-app/run-insights-sync.sh`
    * reconstruir/recrear `spark-client` para refrescar permisos en imagen/contenedor.

10.10 Cambios recientes (30/03/2026)

- Airflow:
  - Corregidas `smoke_query_weather` y `smoke_query_delay` en healthchecks diario/horario para evitar caida cuando faltan tablas streaming fuente.

  - Runs historicos `failed` limpiados para `kdd_daily_healthcheck` y `kdd_hourly_healthcheck`.
  - Auto-recuperacion de meteo Hive en healthchecks (`repopulate_hive_weather_if_empty`) cuando `v_weather_observations_madrid` queda en `0` filas.
- NiFi:
  - Nuevo bootstrap visual por subflujos: `gps_ingestion` y `weather_ingestion`.
  - Proceso de limpieza legacy disponible en `scripts/cleanup_nifi_legacy_pgs.py`.
- Arranque:
  - `scripts/start_kdd.sh` y `scripts/bootstrap_nifi_flow.sh` apuntan por defecto a `kdd_ingestion_auto_v9`.
  - `scripts/start_kdd.sh` repuebla meteo en Hive desde Cassandra si la vista Madrid esta vacia.
- Spark + Dashboard:
  - El dashboard usa Cassandra como fuente primaria de clima (`transport.weather_observations_recent`) con fallback a `nifi/raw-archive/weather`.
  - `LogisticsAnalyticsJob` en modo streaming persiste observaciones meteo recientes en Cassandra para mejorar precision de ruta y panel de clima.
  - `LogisticsAnalyticsJob` en modo streaming persiste tambien `transport_analytics.enriched_events_` en Hive (ademas de `delay_metrics_streaming` y `weather_observations_streaming`).
  - `spark-app/run-streaming.sh` y `spark-app/run-batch.sh` recompilan el JAR automaticamente si hay cambios en `src/` o `pom.xml`.
  - Ajuste de ETA en tiempo real: resincronizacion inmediata cuando hay divergencia grande entre ETA cacheado y ETA fisico (distancia/velocidad) o cambio de destino estimado.
- GPS/NiFi:
  - El generador GPS usa rutas realistas derivadas del grafo logistico y ya no etiquetas aleatorias fijas.
  - Flota leida desde `data/master/vehicles.csv` (IDs actuales `TRUCK-*`; filtra por `status` activo).
  - En NiFi se evita sobrescritura de raw GPS: cada split se archiva con nombre unico (`${filename}_${fragment.index}_${UUID()}.jsonl`).

10.11 Notas

- La arquitectura Hadoop es de nodo unico en modo pseudodistribuido.
- El proyecto prioriza reproducibilidad local y separa claramente batch, streaming y orquestacion.
- Se contempla ingesta meteo (API publica) hacia `transport.weather.raw` y `transport.weather.filtered`, con persistencia en Hive.
- Hay documentacion operativa en `docs/architecture.md`, `docs/nifi-flow.md`, `docs/nifi-weather-templat` y `docs/hive-validation.md`.
- Guia operativa consolidada: `docs/operations.md`.
- Guia de dashboard y leyenda visual: `docs/dashboard.md`.
- Endpoint debug de fuentes del dashboard: `GET /api/debug/sources`.
- Plan TODO de cumplimiento del enunciado por fases KDD: `docs/kdd-todo.md`.
- Bitacora de cambios de la iteracion 30/03/2026: `docs/release-notes-2026-03-30.md`.

- Memoria tecnica detallada del sistema (entrega): `docs/memoria-tecnica-sistema.md`.
- Resumen ejecutivo (1-2 paginas) para portada/anexo inicial: `docs/resumen-ejecutivo-memoria.md`.

11 Changelog

Todos los cambios relevantes de este proyecto se documentan en este archivo.

El formato sigue Keep a Changelog.

11.1 [Unreleased]

11.1.1 Added

- Persistencia streaming en Hive de eventos enriquecidos: `transport_analytics.enriched_events_streaming`.
- Nuevo fallback Parquet de streaming: `/data/curated/enriched_events_streaming`.
- Persistencia de snapshots de insights en Cassandra: `transport.network_insights_snapshots`.
- Nuevo endpoint de historico de insights: `/api/network/insights/history`.
- Nuevo launcher Spark: `spark-app/run-insights-sync.sh`.
- Nuevas tablas Hive de consolidacion de insights:
  - `transport_analytics.network_insights_snapshots_hive`
  - `transport_analytics.network_insights_hourly_trends`
- Script `scripts/rebuild_graph_edges_by_proximity.py` para regenerar aristas por cercania geografica con reglas de negocio.

11.1.2 Changed

- Generador GPS actualizado a rutas realistas sobre grafo logistico y flota actual leida desde `data/master/vehicles.csv`.
- Bootstrap NiFi actualizado a `kdd_ingestion_auto_v9`.
- Flujo GPS NiFi actualizado con renombrado unico por split (`${filename}_${fragment.index}_${UUID()}.js` para evitar sobrescrituras en `raw-archive/gps`.
- Parseo de timestamps ISO UTC robusto en Spark streaming para GPS y clima.
- Documentacion tecnica y operativa actualizada para reflejar el estado real de `v9` y las nuevas salidas streaming.
- Dashboard actualizado:
  - tablas ordenables por columna,
  - selectores de origen/destino ordenados alfabeticamente,
  - etiquetas de nodos RT en codigo de 3 letras y semitransparentes.
- Red logistico-geografica consolidada:
  - 15 nodos (`ACO, ALM, BCN, BIO, CAC, GIJ, LIS, MAD, MAL, MUR, OPO, SEV, VAL, VLL, ZAR`),
  - 15 vehiculos (`14` activos y `1` en mantenimiento),
  - restricciones de conectividad por corredor y atajos prohibidos.

11.1.3 Fixed

- Correccion de sobrescritura de ficheros GPS en `nifi/raw-archive/gps` al archivar flowfiles de `SplitText`.
- Correccion de `UNKNOWN` en congestion de insights cuando no hay muestras live (normalizacion a `low/medium/high`).
- Correccion en computo de insights por perfil (`KeyError: profile_minutes_sum`).

11.1.4 Removed

- Sin cambios todavia.

11.2 [1.0.0] - 2026-03-30

11.2.1 Added

- Script `scripts/repopulate_hive_weather_from_cassandra.sh` para repoblar `transport_analytics.weath` desde Cassandra (`transport.weather_observations_recent`).
- Tarea `repopulate_hive_weather_if_empty` en los DAGs:
  - `dags/kdd_hourly_healthcheck_dag.py`
  - `dags/kdd_daily_healthcheck_dag.py`

11.2.2 Changed

- `scripts/start_kdd.sh` incluye repoblado automatico de meteo en Hive cuando `v_weather_observations_mad` devuelve `weather_rows=0`.
- `docs/operations.md` actualizado con flujo de diagnostico y repoblado manual.
- `README.md` actualizado con el flujo de auto-recuperacion meteo.

11.2.3 Fixed

- ETA incoherente en dashboard por cache/suavizado:
  - resincronizacion forzada al cambiar destino estimado,
  - resincronizacion forzada ante divergencia grande entre ETA cacheado y ETA fisico (`distancia/velocidad + delay`).
  - archivo afectado: `dashboard/static/app.js`.

11.2.4 Removed

- Sin cambios.

11.2.5 Documentation

- Normalizacion de enlaces en `docs/CODE-ANNOTATION-GUIDE.md` a rutas relativas.
- Actualizacion de:
  - `docs/dashboard.md` (logica ETA y estabilidad),
  - `docs/release-notes-2026-03-30.md`,

- `docs/operations.md` (troubleshooting ETA y Hive meteo),
- `README.md` (operacion consolidada).